

# Department of Mechanical Engineering



## Computational Engineering - Lab Report

Muhammad Zakariya Tariq

Registration No. **2023-ME-63**

Section-B

Submitted to: **Sir Waqar Nasir**

# Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>MATLAB Basics, Numerical Errors, Convergence and Plotting</b> | <b>3</b>  |
| <b>2</b> | <b>Root Finding – Bisection Method</b>                           | <b>14</b> |
| <b>3</b> | <b>Root Finding – False Position Method</b>                      | <b>21</b> |
| <b>4</b> | <b>Newton–Raphson Method and Fixed-Point Iteration Method</b>    | <b>30</b> |
| <b>5</b> | <b>Secant Method and Modified Secant Method</b>                  | <b>41</b> |
| <b>6</b> | <b>Inverse Quadratic Interpolation and Newton–Raphson Method</b> | <b>53</b> |

## **MATLAB codes of all Lab sessions**

All MATLAB codes, generated figures, and supplementary data for every lab session reported in this document are publicly available in the following GitHub repository

**Repository URL:**

[https://github.com/mztcoding/CE\\_lab\\_MATLAB](https://github.com/mztcoding/CE_lab_MATLAB)

## Lab Session 1

# MATLAB Basics, Numerical Errors, Convergence and Plotting

### 1.1 Objective

- Introduce the MATLAB programming environment including its key components: Command Window, Editor, Workspace, and Current Folder.
- Develop proficiency in defining variables, vectors, matrices, and implementing control structures such as `for` and `while` loops and conditional statements.
- Generate 2D and 3D plots, subplots, and histograms using MATLAB's built-in visualization tools.
- Understand and quantify numerical approximation errors by studying the convergence of the Maclaurin series expansion of  $\sin(x)$  with an increasing number of terms.

### 1.2 Introduction and Theoretical Background

#### 1.2.1 MATLAB Environment

MATLAB (MATrix LABoratory) is a high-level programming language designed for numerical computation, algorithm development, and data visualisation [1, 2]. Its matrix-oriented syntax makes it particularly suited to engineering and scientific computing.

The key interface components are the **Command Window** (interactive execution), **Editor** (script and function authoring), **Workspace** (variable management), and **Current Folder** (file access). Scripts typically begin with the following three commands to ensure a clean environment:

```
clc;           % Clear command window
clear;        % Remove workspace variables
close all     % Close all figure windows
```

#### 1.2.2 Core Language Features

All data in MATLAB is stored as matrices; scalars and vectors are special cases. Element-wise operations use dot notation (`.*`, `./`, `.^`). Variables require no type declaration, and functions are defined with the `function` keyword. The language supports standard

control structures including `for/while` loops and `if/else` conditionals, which form the basis of iterative numerical algorithms.

```
x = 0:0.1:10;           % Row vector
z = x.^2 + 3.*x;        % Element-wise operations

function y = squareVal(x) % User-defined function
    y = x.^2;
end

for i = 1:5              % Loop
    disp(i);
end

if x > 0                  % Conditional
    disp('Positive');
else
    disp('Non-positive');
end
```

### 1.2.3 Plotting and Visualisation

MATLAB provides built-in functions for generating annotated 2D and 3D plots. The `plot` function is used for standard line graphs, `semilogy` for logarithmic-scale convergence plots, `subplot` for multi-panel figures, and `surf/plot3` for three-dimensional visualisation.

```
x = 0:0.1:2*pi;
plot(x, sin(x), 'b-', 'LineWidth', 1.5);
xlabel('x'); ylabel('sin(x)');
title('Sine Function'); grid on;
```

### 1.2.4 Running Code and Saving Results

MATLAB scripts are executed with F5 or by typing the script name in the Command Window; selected lines run with F9. Output uses `fprintf` or `disp`, and a semicolon suppresses it. Variables can be saved to `.mat` files and results exported to text or spreadsheet files.

```
fprintf('Root: x = %.6f (iterations: %d)\n', x, n); %
    formatted print
save('results.mat', 'x', 'n');                    % save variables to .
    mat
load('results.mat');                               % reload in a new
    session
writematrix(data, 'output.xlsx');                  % export to
    spreadsheet
```

### 1.2.5 Numerical Methods in MATLAB

Numerical methods provide approximate solutions to mathematical problems that are difficult to solve analytically. MATLAB's matrix-based environment, built-in functions, and visualization tools make it ideal for implementing these methods [3].

Key techniques include:

- **Root-Finding:** Methods like Newton-Raphson and Fixed-Point iteration for solving nonlinear equations.
- **Numerical Integration/Differentiation:** Approximating derivatives and integrals using finite differences, trapezoidal, or Simpson's rule.
- **Linear Systems:** Solving systems of equations with matrix operations and the backslash operator.
- **Series Approximation:** Using Maclaurin or Taylor series to approximate functions and study convergence.
- **Error Analysis:** Quantifying approximation errors and studying convergence in iterative methods.

MATLAB allows efficient coding of these methods, visualization of results, and analysis of numerical accuracy and stability.

### 1.2.6 Taylor Series and Numerical Errors

Any sufficiently smooth function  $f(x)$  can be represented as an infinite power series expanded about a point  $a$  [4, 3]. When  $a = 0$ , the series is called the **Maclaurin series**. For the sine function, the Maclaurin series is:

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots \quad (1.1)$$

where  $x$  must be expressed in radians. A finite truncation of this series to  $N$  terms introduces a **truncation error**. Three standard error metrics are defined as follows:

$$\text{True Error: } E_t = f_{\text{true}} - f_{\text{approx}} \quad (1.2)$$

$$\text{Absolute Error: } |E_t| \quad (1.3)$$

$$\text{Relative Error: } \varepsilon_t = \frac{|E_t|}{|f_{\text{true}}|} \quad (1.4)$$

As the number of terms  $N$  increases, the approximation converges to the true value and the errors decrease. Plotting the error on a semi-logarithmic scale illustrates the rate of convergence clearly. For this lab session, convergence is studied at  $x = 60^\circ$  (i.e.,  $x = \pi/3$  radians).

## 1.3 Methodology

The lab session is conducted in MATLAB through the following steps:

1. Define the domain as  $x \in [-360^\circ, 360^\circ]$  in steps of  $1^\circ$  and convert to radians using `deg2rad`.
2. Compute the true value  $\sin(x)$  using MATLAB's built-in `sin` function as the reference.
3. Implement a nested loop to compute the Maclaurin series approximation for  $N = 1, 2, \dots, 10$  terms, storing each approximation in a row of a matrix `T`.
4. Evaluate the series at  $x = 60^\circ$  to study pointwise convergence with respect to the number of terms.
5. Compute true error, absolute error, and relative error at  $x = 60^\circ$  for each  $N$ .
6. Generate four plots: (i) the true  $\sin(x)$  alongside all ten approximations over the full domain; (ii) approximated and true values at  $x = 60^\circ$  versus number of terms; (iii) absolute error versus number of terms on a semi-log scale; and (iv) relative error versus number of terms on a semi-log scale.

## 1.4 Algorithm

1. Set the number of terms  $N_{\max} = 10$ .
2. Define  $x_{\text{deg}} = [-360, -359, \dots, 360]$  and convert to radians:  $x = x_{\text{deg}} \times \pi/180$ .
3. Compute the true function:  $f_{\text{true}} = \sin(x)$ .
4. **For**  $N = 1$  to  $N_{\max}$ :
  - (a) Initialise partial sum  $T_{\text{sum}} = 0$ .
  - (b) **For**  $n = 0$  to  $N - 1$ :  $T_{\text{sum}} += \frac{(-1)^n}{(2n+1)!} x^{2n+1}$
  - (c) Store  $T(N, :) = T_{\text{sum}}$ .
5. Plot  $f_{\text{true}}$  and all rows of  $T$  against  $x_{\text{deg}}$  on the same axes.
6. Set  $x_{60} = \pi/3$ . Compute the true value  $f_{60} = \sin(x_{60})$ .
7. Evaluate the series for  $N = 1, \dots, N_{\max}$  at  $x_{60}$  to obtain the array `approx_val`.
8. Compute:  $E_t = f_{60} - \text{approx\_val}$ ,  $|E_t|$ , and  $\varepsilon_t = |E_t|/|f_{60}|$ .
9. Plot convergence, absolute error, and relative error versus  $N$ .

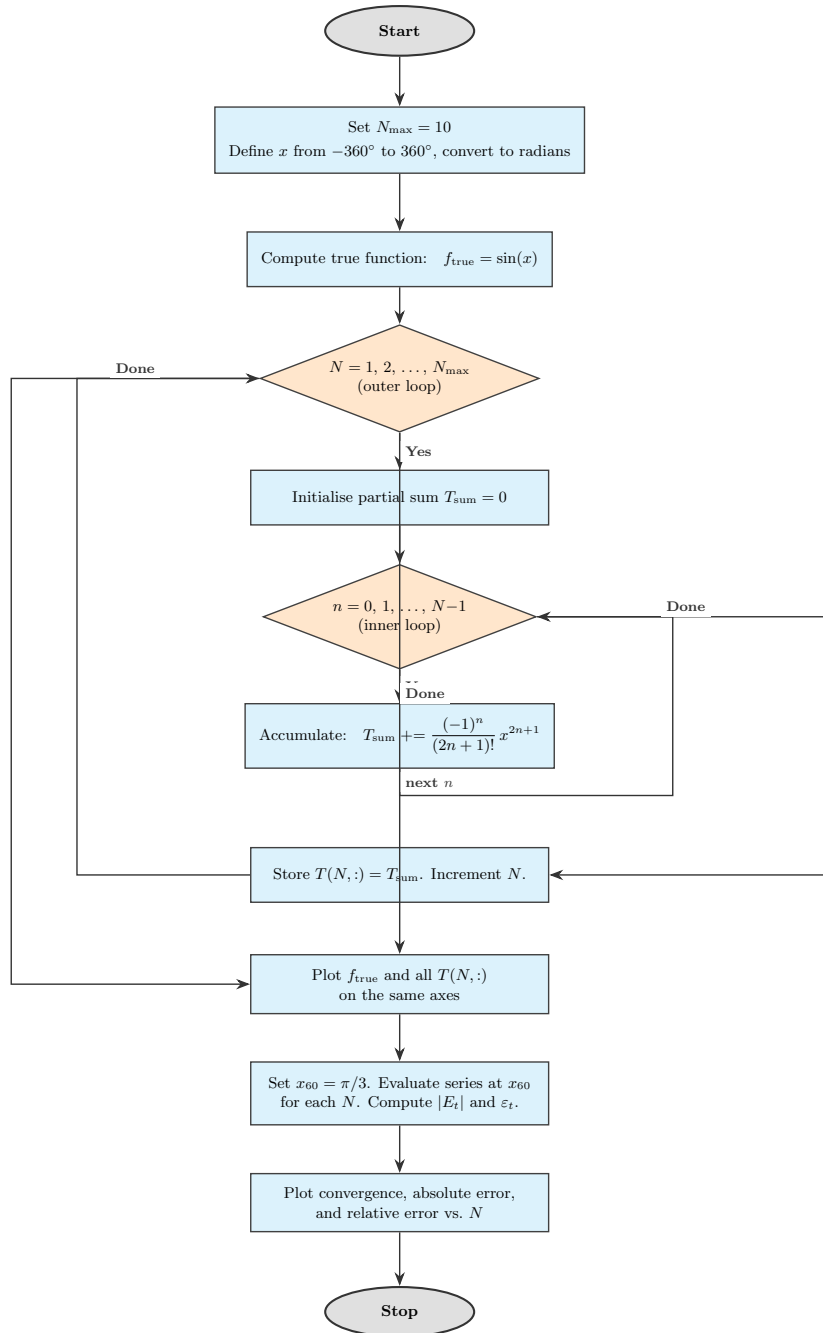


Figure 1.1: Flowchart of the Taylor series algorithm for  $\sin(x)$ : nested loops, error computation, and plotting.

## 1.5 MATLAB Implementation

This section presents the complete MATLAB codes for the three problems covered in Lab Session 1: (i) solving the quadratic equation, (ii) the falling parachutist problem (analytical and numerical), and (iii) the Taylor series approximation of  $\sin(x)$  with convergence and error analysis.



## Problem 1: Quadratic Equation Solver

The roots of  $ax^2 + bx + c = 0$  are computed using the quadratic formula, with handling for complex roots and the case  $a = 0$ .

```
1 clc; clear; close all;
2
3 while true
4     % ----- User input -----
5     a = input('Enter coefficient a: ');
6     b = input('Enter coefficient b: ');
7     c = input('Enter coefficient c: ');
8
9     % ----- Special case: linear equation -----
10    if a == 0
11        if b == 0
12            disp('Not an equation (a = b = 0).');
13        else
14            fprintf('Linear equation. Root: x = %.6f\n', -c/b);
15        end
16    else
17        % ----- Discriminant -----
18        disc = b^2 - 4*a*c;
19
20        if disc > 0
21            x1 = (-b + sqrt(disc)) / (2*a);
22            x2 = (-b - sqrt(disc)) / (2*a);
23            fprintf('Two real roots:\n  x1 = %.6f\n  x2 = %.6f\n', x1, x2);
24        elseif disc == 0
25            x1 = -b / (2*a);
26            fprintf('One repeated real root: x = %.6f\n', x1);
27        else
28            realPart = -b / (2*a);
29            imagPart = sqrt(-disc) / (2*a);
30            fprintf('Complex roots:\n');
31            fprintf('  x1 = %.6f + %.6fi\n', realPart, imagPart);
32            fprintf('  x2 = %.6f - %.6fi\n', realPart, imagPart);
33        end
34    end
35
36    % ----- Repeat? -----
37    again = input('\nSolve another equation? (1 = Yes, 0 = No): ');
38    if again ~= 1
```

```

39         break;
40     end
41 end

```

Code 1.1: Quadratic equation solver with user input and loop.

## Problem 2: Falling Parachutist – Analytical and Numerical Solutions

A parachutist of mass  $m = 68.1$  kg and drag coefficient  $c = 12.5$  kg/s is released from rest. The analytical velocity is:

$$v(t) = \frac{gm}{c} (1 - e^{-(c/m)t}) \quad (1.5)$$

The Euler numerical approximation uses a step size of  $\Delta t = 2$  s:

$$v_{i+1} = v_i + \left(g - \frac{c}{m}v_i\right) \Delta t \quad (1.6)$$

For the second-order drag model (Problem 1.3), the upward drag force is  $F_U = -c'v^2$  with  $c' = 0.225$  kg/m.

```

1  clc; clear; close all;
2
3  %% Parameters
4  m = 68.1; c = 12.5; g = 9.8; dt = 2; t_end = 14;
5  c2 = 0.225; % 2nd-order drag coefficient (kg/m)
6
7  %% Analytical solution
8  t_anal = 0:0.1:t_end;
9  v_anal = (g*m/c) .* (1 - exp(-(c/m).*t_anal));
10
11 %% Euler -- linear drag
12 t_num = 0:dt:t_end;
13 v_num = zeros(size(t_num));
14 for i = 1:length(t_num)-1
15     v_num(i+1) = v_num(i) + (g - (c/m)*v_num(i)) * dt;
16 end
17
18 %% Euler -- second-order drag
19 v_num2 = zeros(size(t_num));
20 for i = 1:length(t_num)-1
21     v_num2(i+1) = v_num2(i) + (g - (c2/m)*v_num2(i)^2) * dt;
22 end
23
24 %% Plot
25 figure;
26 plot(t_anal, v_anal, 'k-', 'LineWidth', 2, 'DisplayName', '
    Analytical'); hold on;

```

```

27 plot(t_num,v_num,'bo-','MarkerFaceColor','b','DisplayName','
    Euler (linear)');
28 plot(t_num,v_num2,'rs-','MarkerFaceColor','r','DisplayName','
    Euler (2nd-order)');
29 yline(g*m/c,'k--','DisplayName','Terminal velocity');
30 xlabel('Time (s)'); ylabel('Velocity (m/s)');
31 title('Falling Parachutist'); legend('Location','southeast');
    grid on;

```

Code 1.2: Analytical and numerical (Euler) solutions for the falling parachutist problem.

### Problem 3: Taylor Series Approximation of $\sin(x)$

```

1  clc; clear; close all;
2
3  %% Build Maclaurin approximations (N = 1..10 terms)
4  Nterms = 10;
5  x = deg2rad(-360:1:360); f_true = sin(x);
6  T = zeros(Nterms, length(x));
7  for N = 1:Nterms
8      s = zeros(size(x));
9      for n = 0:N-1
10         s = s + ((-1)^n/factorial(2*n+1)) .* x.^(2*n+1);
11     end
12     T(N,:) = s;
13 end
14
15 %% Plot 1 -- all approximations vs true sin(x)
16 figure; plot(rad2deg(x), f_true,'k','LineWidth',3,'
    DisplayName','True sin(x)'); hold on;
17 for N = 1:Nterms, plot(rad2deg(x),T(N,:),'--','DisplayName',
    sprintf('%d terms',N)); end
18 xlabel('x (deg)'); ylabel('sin(x)'); legend('Location','best'
    ); grid on;
19 xlim([-360 360]); ylim([-1.5 1.5]); title('Maclaurin
    Approximations of sin(x)');
20
21 %% Convergence and error at x = 60 degrees
22 x60 = deg2rad(60); tv = sin(x60);
23 av = zeros(1,Nterms);
24 for N = 1:Nterms
25     for n = 0:N-1, av(N) = av(N) + ((-1)^n/factorial(2*n+1))*
        x60^(2*n+1); end
26 end
27 abs_err = abs(tv - av); rel_err = abs_err/abs(tv);
28
29 %% Plot 2 -- convergence at 60 deg
30 figure; plot(1:Nterms,av,'o-'); hold on; yline(tv,'r--');

```

```

31 xlabel('Terms'); ylabel('sin(60{\circ})'); title('Convergence
    at x=60{\circ}'); grid on;
32
33 %% Plot 3 & 4 -- errors
34 figure; semilogy(1:Nterms,abs_err,'s-'); xlabel('Terms');
    ylabel('Absolute Error'); grid on;
35 figure; semilogy(1:Nterms,rel_err,'d-'); xlabel('Terms');
    ylabel('Relative Error'); grid on;

```

Code 1.3: Maclaurin series approximation of  $\sin(x)$ , convergence study and error analysis.

## 1.6 Results and Discussion

### 1.6.1 Falling Parachutist: Analytical vs. Numerical Solution

The analytical solution describes the exact velocity of the parachutist under linear drag, while the Euler numerical method provides an approximate solution using discrete time steps. Both approaches model the same physical motion and show similar trends in velocity as the parachutist accelerates and approaches terminal velocity.

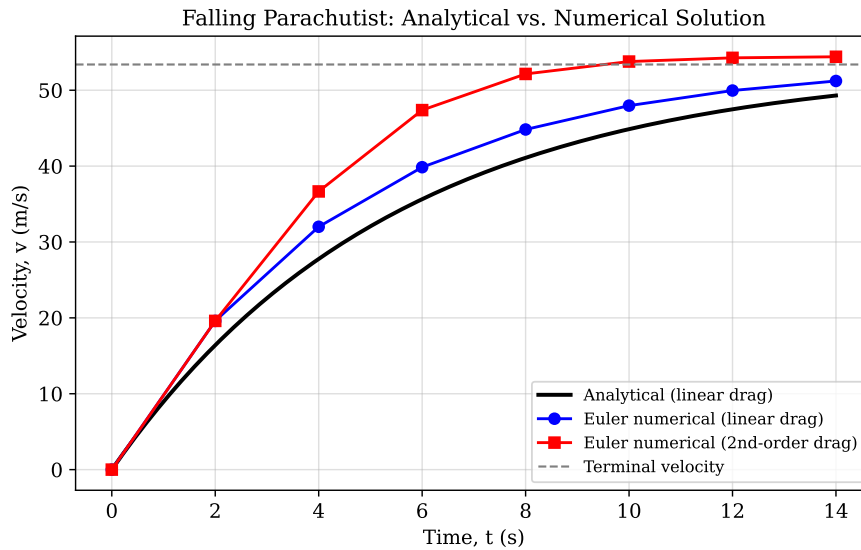


Figure 1.2: Falling parachutist: analytical solution (linear drag), Euler numerical solution (linear drag), and Euler numerical solution (second-order drag,  $c' = 0.225$  kg/m).

### 1.6.2 Taylor Series Approximations over the Full Domain

The first plot displays the true  $\sin(x)$  function alongside all ten Maclaurin approximations over  $[-360^\circ, 360^\circ]$ . With only one or two terms the series closely matches the true function only in the vicinity of the origin. As  $N$  increases, the interval over which the approximation remains accurate widens progressively. Higher-order approximations eventually agree with the true curve over nearly the entire plotted domain, illustrating the well-known property that the Maclaurin series for  $\sin(x)$  has an infinite radius of convergence.

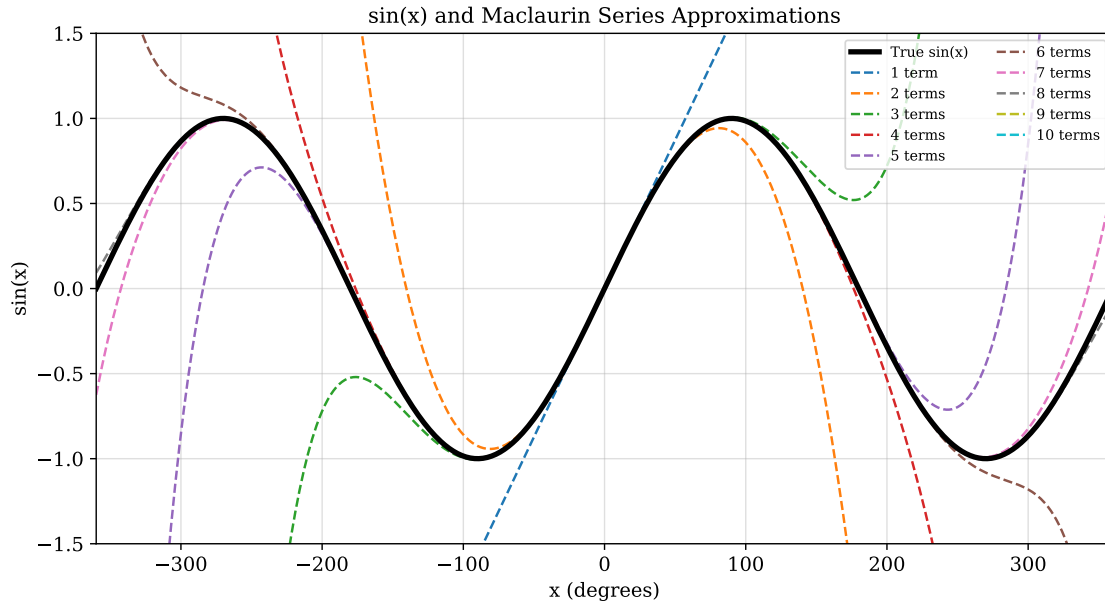


Figure 1.3: True  $\sin(x)$  and Maclaurin series approximations for  $N = 1$  to 10 over  $[-360^\circ, 360^\circ]$ .

### 1.6.3 Convergence at $x = 60^\circ$

The second plot shows how the series approximation at  $x = 60^\circ$  approaches the true value of  $\sin(60^\circ) = \sqrt{3}/2 \approx 0.8660$  as more terms are included. With only one term ( $\sin(x) \approx x$ ) the approximation is noticeably poor; by approximately  $N = 5$  the approximation is visually indistinguishable from the true value.

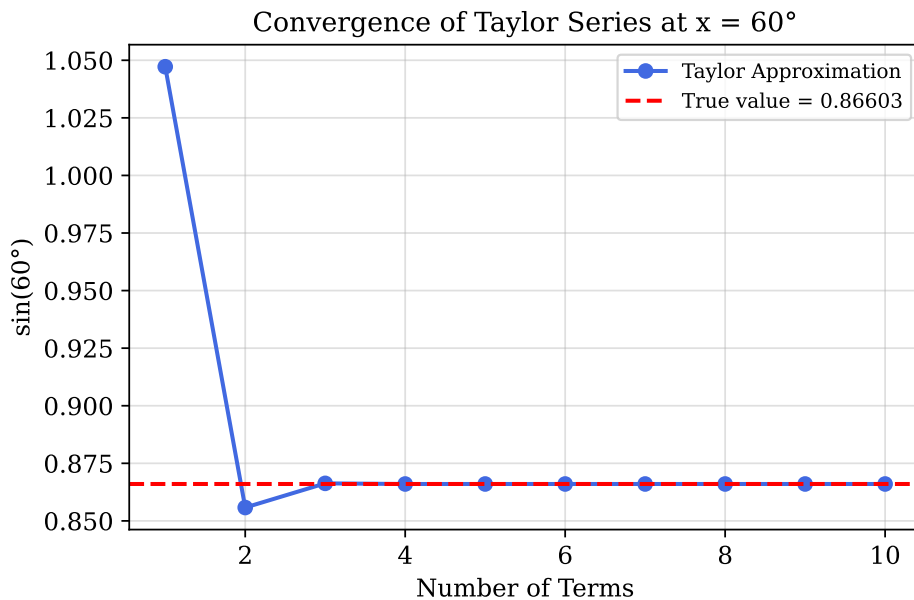


Figure 1.4: Convergence of the Maclaurin approximation at  $x = 60^\circ$  as a function of the number of terms.

### 1.6.4 Error Analysis

Plots of absolute and relative error on semi-logarithmic axes confirm that both errors decrease monotonically with increasing  $N$ . The rapid, approximately exponential decay of the error reflects the factorial growth of the denominators in the series coefficients, which causes each successive term to contribute less than the previous one. This behaviour is characteristic of a **geometrically converging** series near the expansion point.

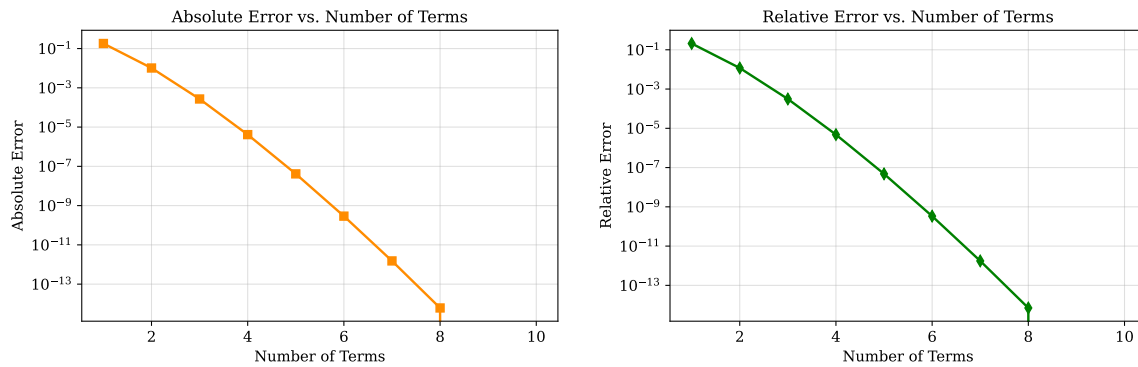


Figure 1.5: Absolute and relative error versus number of Maclaurin terms at  $x = 60^\circ$  (semi-log scale).

The results provide a detailed insight into the behaviour, accuracy, and limitations of the numerical approaches implemented in this lab. The graphical comparison for the parachutist problem illustrates how discretisation in Euler's method introduces noticeable deviation from the analytical solution, particularly at larger time steps, while also showing the influence of different drag models on system response. In the Taylor series study, the progressive improvement in approximation with increasing terms demonstrates how convergence is strongly dependent on both the number of terms and the distance from the expansion point. The semi-log error plots further reveal a consistent and rapid reduction in error, indicating stable numerical behaviour and reinforcing the theoretical expectations of series convergence. These observations collectively highlight the importance of step size selection, model formulation, and convergence assessment when applying numerical techniques to engineering problems.

## 1.7 Conclusion

This lab session established familiarity with the MATLAB environment and its core programming constructs. Students practised defining arrays, implementing nested loops, creating user-defined functions, and generating annotated plots. The study of the Maclaurin series for  $\sin(x)$  demonstrated that truncating an infinite series introduces a quantifiable truncation error, and that this error diminishes as more terms are retained. The parachutist problem illustrated the difference between analytical and numerical (Euler) solutions for a first-order ODE and the effect of using a first-order versus second-order drag model. Overall, the lab strengthened practical implementation skills in numerical computing and reinforced the structured approach required to translate mathematical formulations into efficient computational algorithms.

## Lab Session 2

# Root Finding – Bisection Method

## 2.1 Objective

- Implement the bisection method in MATLAB to locate real roots of nonlinear equations of the form  $f(x) = 0$ .
- Study the influence of the initial bracketing interval and convergence tolerance on the number of iterations required.
- Validate the computed root graphically by plotting the function and marking the successive root approximations.
- Visualise the convergence behaviour by plotting the approximate percentage relative error against the iteration count.

## 2.2 Introduction and Theoretical Background

### 2.2.1 Root-Finding Problem

Many engineering problems reduce to solving a nonlinear equation of the form [4, 5]:

$$f(x) = 0 \quad (2.1)$$

Closed-form analytical solutions are often unavailable, making iterative numerical methods necessary. Such methods are broadly classified as either **bracketing methods**, which require an initial interval  $[x_l, x_u]$  that is known to contain a root, or **open methods**, which require only an initial guess.

### 2.2.2 Bisection Method

The bisection method is the simplest and most robust bracketing technique [4, 3]. It relies on the **Intermediate Value Theorem**: if  $f(x_l)$  and  $f(x_u)$  have opposite signs, i.e.  $f(x_l) \cdot f(x_u) < 0$ , then at least one root exists in the interval  $[x_l, x_u]$ .

At each iteration, the midpoint is computed:

$$x_r = \frac{x_l + x_u}{2} \quad (2.2)$$

and the subinterval containing the root is identified by evaluating the sign of  $f(x_r)$ :

$$\begin{cases} \text{Replace } x_u \text{ with } x_r & \text{if } f(x_l) \cdot f(x_r) < 0 \\ \text{Replace } x_l \text{ with } x_r & \text{if } f(x_r) \cdot f(x_u) < 0 \\ x_r \text{ is the root} & \text{if } f(x_r) = 0 \end{cases} \quad (2.3)$$

The convergence of the bisection method is guaranteed for continuous functions satisfying the sign-change criterion. The approximate percentage relative error at each iteration is:

$$\varepsilon_a = \left| \frac{x_r^{\text{new}} - x_r^{\text{old}}}{x_r^{\text{new}}} \right| \times 100\% \quad (2.4)$$

Iteration continues until  $\varepsilon_a < \varepsilon_s$ , where  $\varepsilon_s$  is the specified stopping criterion. In this lab session,  $\varepsilon_s = 0.01\%$ .

The method halves the interval at every step, so after  $n$  iterations the maximum possible error in the root is:

$$\Delta x_n = \frac{x_u - x_l}{2^n} \quad (2.5)$$

This linear (first-order) convergence rate is reliable but slower than higher-order methods discussed in later sessions.

## 2.3 Methodology

1. Define the nonlinear function  $f(x)$  as an anonymous function handle in MATLAB.
2. Prompt the user to enter the bracketing points  $x_l$  and  $x_u$  via `input()`.
3. Verify the sign-change condition  $f(x_l) \cdot f(x_u) < 0$  before proceeding.
4. Implement the bisection iteration inside a **while** loop, terminating when  $\varepsilon_a < 0.01\%$  or a maximum iteration count is reached.
5. Record  $x_r$  and  $\varepsilon_a$  at each iteration in arrays for post-processing.
6. Plot the function over an appropriate interval, marking each successive root estimate with its iteration number directly on the graph.
7. Plot the approximate percentage relative error versus iteration number to visualise convergence.

## 2.4 Algorithm

1. Define  $f(x)$ .
2. Input lower bound  $x_l$  and upper bound  $x_u$  from the user.
3. Check: if  $f(x_l) \cdot f(x_u) \geq 0$ , display an error message and terminate.
4. Initialise: iteration counter  $i = 0$ ,  $\varepsilon_a = \infty$ ,  $x_r^{\text{old}} = x_l$ .
5. **While**  $\varepsilon_a > \varepsilon_s$  **and**  $i < i_{\text{max}}$ :
  - (a)  $i \leftarrow i + 1$ .
  - (b)  $x_r \leftarrow (x_l + x_u)/2$ .
  - (c) Compute  $\varepsilon_a = |(x_r - x_r^{\text{old}})/x_r| \times 100\%$ .
  - (d) If  $f(x_l) \cdot f(x_r) < 0$ : set  $x_u \leftarrow x_r$ .



- (e) Else if  $f(x_r) \cdot f(x_u) < 0$ : set  $x_l \leftarrow x_r$ .
  - (f) Else: root found exactly; break.
  - (g)  $x_r^{\text{old}} \leftarrow x_r$ .
  - (h) Store  $x_r$  and  $\varepsilon_a$ .
6. Report the root  $x_r$  and total iteration count.
  7. Generate the two required plots.

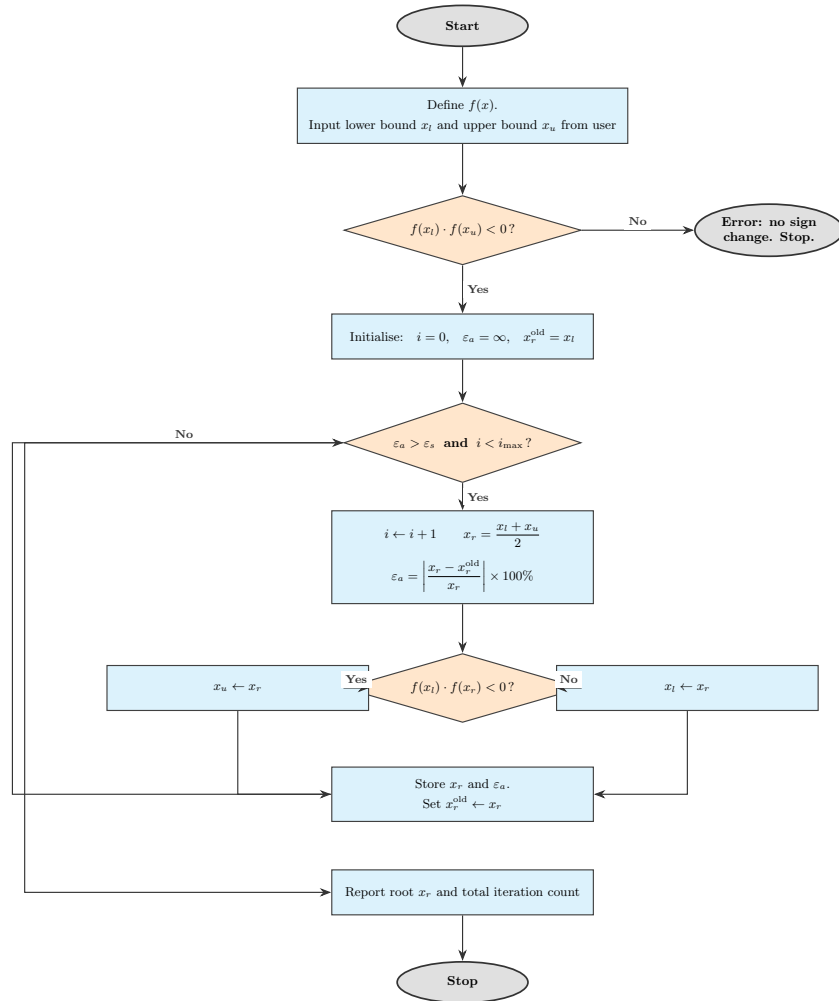


Figure 2.1: Flowchart of the Bisection Method algorithm.

## 2.5 MATLAB Implementation

Two problems from Chapra & Canale are solved using the bisection method. **Problem 1** (Example 5.1) finds the drag coefficient  $c$  required for a parachutist of mass  $m = 68.1$  kg to reach  $v = 40$  m/s after  $t = 10$  s of free fall. **Problem 2** (Example 5.2) locates a root of  $f(x) = \sin 10x + \cos 3x$  on  $[0, 5]$ .

## Problem 1: Parachutist Drag Coefficient

The governing equation to be solved is:

$$f(c) = \frac{9.8 \times 68.1}{c} (1 - e^{-(c/68.1) \times 10}) - 40 = 0 \quad (2.6)$$

A sign change is observed between  $c = 12$  and  $c = 16$ .

```
1 %% Problem definition
2 m = 68.1; g = 9.8; v_target = 40; t = 10;
3 f = @(c) (g*m./c).*(1 - exp(-(c/m)*t)) - v_target;
4 tol = 0.0001; max_iter = 100;
5
6 xl = input('Enter lower bracket xl: ');
7 xu = input('Enter upper bracket xu: ');
8 if f(xl)*f(xu) >= 0, error('No sign change in bracket.');
```

---

```
10 %% Bisection iteration
11 xr_old = xl; ea = inf; roots_arr = []; ea_arr = []; iter = 0;
12 while ea > tol && iter < max_iter
13     iter = iter + 1;
14     xr = (xl + xu) / 2;
15     if iter > 1, ea = abs((xr - xr_old)/xr)*100; end
16     roots_arr(end+1) = xr; ea_arr(end+1) = ea;
17     if f(xl)*f(xr) < 0, xu = xr;
18     elseif f(xr)*f(xu) < 0, xl = xr;
19     else, break; end
20     xr_old = xr;
21 end
22 fprintf('Root: c = %.6f (%d iterations)\n', xr, iter);
```

Code 2.1: Bisection method applied to the parachutist drag coefficient problem.

## Problem 2: Roots of $f(x) = \sin 10x + \cos 3x$

This function has several roots on  $[0, 5]$ . The code locates one root in a user-specified sub-interval.

```
1 %% Problem definition
2 f = @(x) sin(10*x) + cos(3*x);
3 tol = 0.0001; max_iter = 100;
4
5 xl = input('Enter lower bracket xl: ');
6 xu = input('Enter upper bracket xu: ');
7 if f(xl)*f(xu) >= 0, error('No sign change in bracket.');
```

---

```
9 %% Bisection iteration
10 xr_old = xl; ea = inf; roots_arr = []; ea_arr = []; iter = 0;
11 while ea > tol && iter < max_iter
12     iter = iter + 1;
```

```

13     xr    = (xl + xu) / 2;
14     if iter > 1, ea = abs((xr-xr_old)/xr)*100; end
15     roots_arr(end+1) = xr;  ea_arr(end+1) = ea;
16     if      f(xl)*f(xr) < 0, xu = xr;
17     elseif f(xr)*f(xu) < 0, xl = xr;
18     else, break; end
19     xr_old = xr;
20 end
21 fprintf('Root: x = %.6f  (%d iterations)\n', xr, iter);

```

Code 2.2: Bisection method applied to  $f(x) = \sin 10x + \cos 3x$ .

## 2.6 Results and Discussion

### 2.6.1 Function Graph and Root Estimates

The function  $f(x)$  is plotted over a suitable interval, and the root approximation at each bisection iteration is marked on the graph. The successive estimates are labelled 1, 2, 3, ... to show how the search progressively narrows about the true root. An appropriate bracketing interval is one where the function changes sign; an invalid interval (no sign change) is rejected by the algorithm before iteration begins.

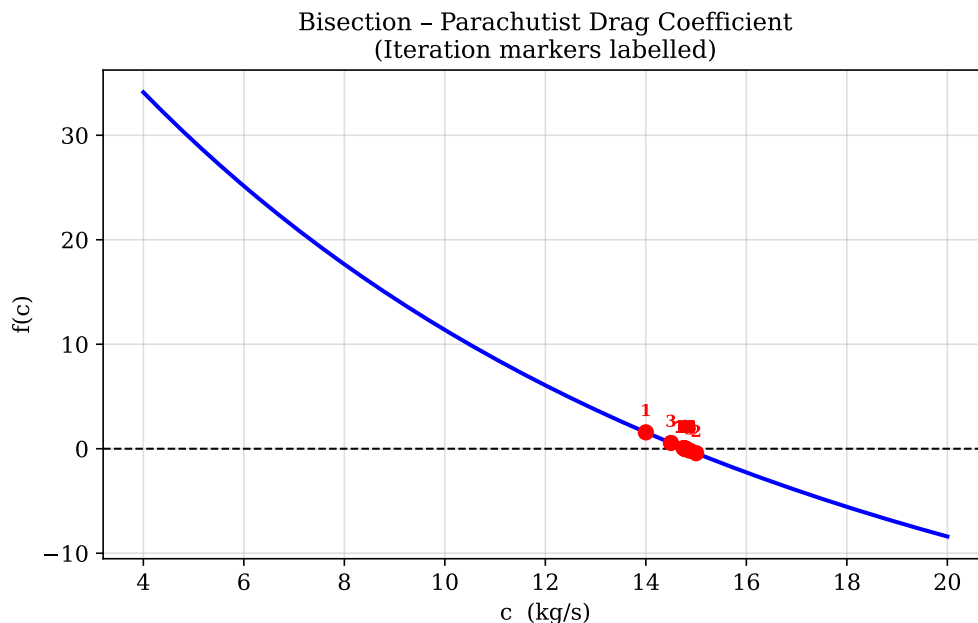


Figure 2.2: Bisection on  $f(c)$  (parachutist drag coefficient): function plot with successive root approximations labelled by iteration number.

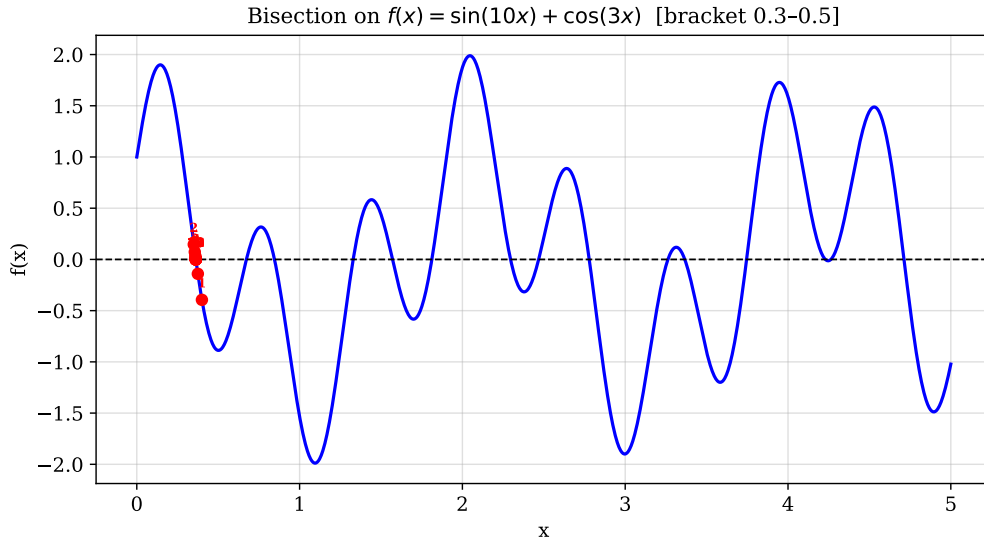


Figure 2.3: Bisection on  $f(x) = \sin 10x + \cos 3x$ : root approximations labelled by iteration number on the function plot.

## 2.6.2 Convergence Plot

The approximate percentage relative error decreases monotonically with each iteration. Because the interval is halved at every step, the error reduces by a constant factor of approximately 50% per iteration, confirming the first-order (linear) convergence rate of the bisection method. Approximately  $n = \lceil \log_2((x_u - x_l)/\varepsilon_s) \rceil$  iterations are needed to achieve a prescribed tolerance, which can be estimated before execution.

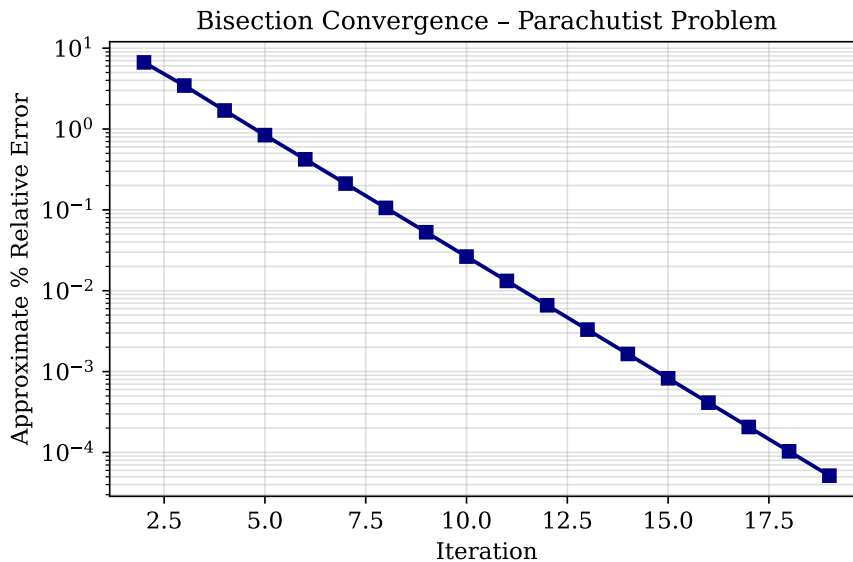


Figure 2.4: Approximate % relative error vs. iteration — bisection on the parachutist drag coefficient problem.

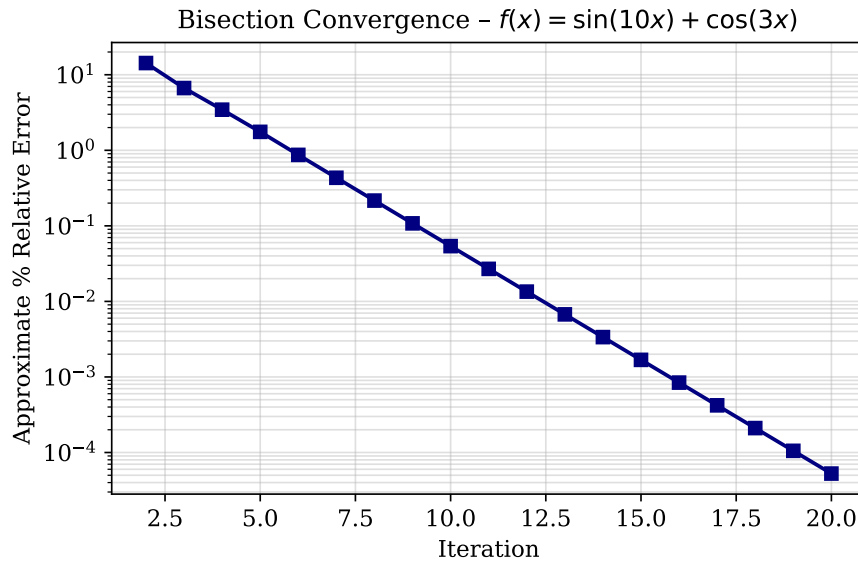


Figure 2.5: Approximate % relative error vs. iteration — bisection on  $f(x) = \sin 10x + \cos 3x$ .

### 2.6.3 Effect of Initial Bracket

A wider initial bracket requires more iterations to achieve the same tolerance, whereas a bracket very close to the true root converges in fewer steps. However, unlike open methods, the bisection method converges regardless of how wide the initial bracket is, provided the sign-change condition is satisfied. This unconditional convergence is the primary advantage of the method.

## 2.7 Conclusion

The bisection method provides a robust and guaranteed means of locating roots of continuous functions, provided a valid bracketing interval is identified. Its linear convergence rate makes it slower than open methods such as Newton–Raphson, but its reliability and simplicity render it valuable as a baseline and as a fallback when other methods fail. The lab session demonstrated how the initial bracket and stopping tolerance directly control the iteration count, and how the error reduces by half at each step in a predictable and systematic manner. In addition, the graphical visualisation of successive root estimates reinforced the conceptual understanding of interval halving and convergence behaviour, making the iterative process more intuitive. The implementation in MATLAB further highlighted how numerical algorithms can be efficiently translated into code for practical engineering applications. Overall, this study emphasises that while the bisection method may not be computationally optimal, its stability, transparency, and guaranteed convergence make it an essential foundational tool in numerical analysis.

## Lab Session 3

# Root Finding – False Position Method

### 3.1 Objective

- Implement the false position (Regula Falsi) method in MATLAB to solve nonlinear equations.
- Compare the convergence speed of the false position method against the bisection method on the same problems.
- Identify and discuss the limitations of the false position method, in particular the phenomenon of one-sided stagnation.
- Visualise the successive root approximations and the relative error convergence for both methods on comparative plots.

### 3.2 Introduction and Theoretical Background

#### 3.2.1 Motivation

The bisection method guarantees convergence but uses only the *sign* of the function at the bracket endpoints, discarding magnitude information. The false position method retains the bracketing structure for reliability while using linear interpolation between the two endpoints to produce a better-informed estimate of the root, often resulting in faster convergence [4, 6].

#### 3.2.2 False Position (Regula Falsi) Method

Like bisection, the false position method maintains a bracket  $[x_l, x_u]$  satisfying  $f(x_l) \cdot f(x_u) < 0$ . However, instead of the arithmetic midpoint, the root estimate is the  $x$ -intercept of the chord (secant line) connecting the points  $(x_l, f(x_l))$  and  $(x_u, f(x_u))$ :

$$x_r = x_u - \frac{f(x_u)(x_l - x_u)}{f(x_l) - f(x_u)} \quad (3.1)$$

This is equivalent to applying the linear interpolation formula. The update rule is identical to bisection: the half of the interval whose endpoint has the same sign as  $f(x_r)$  is discarded and replaced by  $x_r$ .

The approximate percentage relative error is tracked in the same manner as bisection:

$$\varepsilon_a = \left| \frac{x_r^{\text{new}} - x_r^{\text{old}}}{x_r^{\text{new}}} \right| \times 100\% \quad (3.2)$$

### 3.2.3 Limitations – One-Sided Stagnation

A well-known weakness of the false position method is that for strongly curved functions one endpoint of the bracket can remain fixed throughout all iterations, with only the other endpoint being updated [3, 6]. This causes the method to converge from one side only and can result in slower convergence than bisection in such cases. Several modifications (e.g. the Illinois algorithm) address this deficiency by artificially halving the function value at the stagnant endpoint, but these are beyond the scope of the present lab session.

## 3.3 Methodology

1. Define the same nonlinear equation(s) used in the bisection lab session.
2. Accept the initial bracket  $[x_l, x_u]$  from the user, verifying the sign-change condition.
3. Implement the false position formula inside a **while** loop with the same stopping criterion ( $\varepsilon_a < 0.001\%$ ) as employed for bisection.
4. Record the root estimate and error at each iteration.
5. Plot the function graph with successive root approximations marked by iteration number.
6. Plot the relative error versus iteration for both the false position and bisection methods on the same axes for direct comparison.
7. Produce a comparative graph showing the function alongside the successive approximations obtained by both methods simultaneously.

## 3.4 Algorithm

1. Define  $f(x)$ .
2. Input  $x_l$  and  $x_u$ ; verify  $f(x_l) \cdot f(x_u) < 0$ .
3. Initialise:  $i = 0$ ,  $\varepsilon_a = \infty$ ,  $x_r^{\text{old}} = x_l$ .
4. **While**  $\varepsilon_a > \varepsilon_s$  **and**  $i < i_{\text{max}}$ :
  - (a)  $i \leftarrow i + 1$ .
  - (b) Apply false position formula:  $x_r = x_u - \frac{f(x_u)(x_l - x_u)}{f(x_l) - f(x_u)}$ .
  - (c) Compute  $\varepsilon_a = |(x_r - x_r^{\text{old}})/x_r| \times 100\%$ .
  - (d) If  $f(x_l) \cdot f(x_r) < 0$ : set  $x_u \leftarrow x_r$ ; else set  $x_l \leftarrow x_r$ .
  - (e)  $x_r^{\text{old}} \leftarrow x_r$ .
  - (f) Store  $x_r$  and  $\varepsilon_a$ .
5. Report the converged root and number of iterations.
6. Generate the required individual and comparative plots.

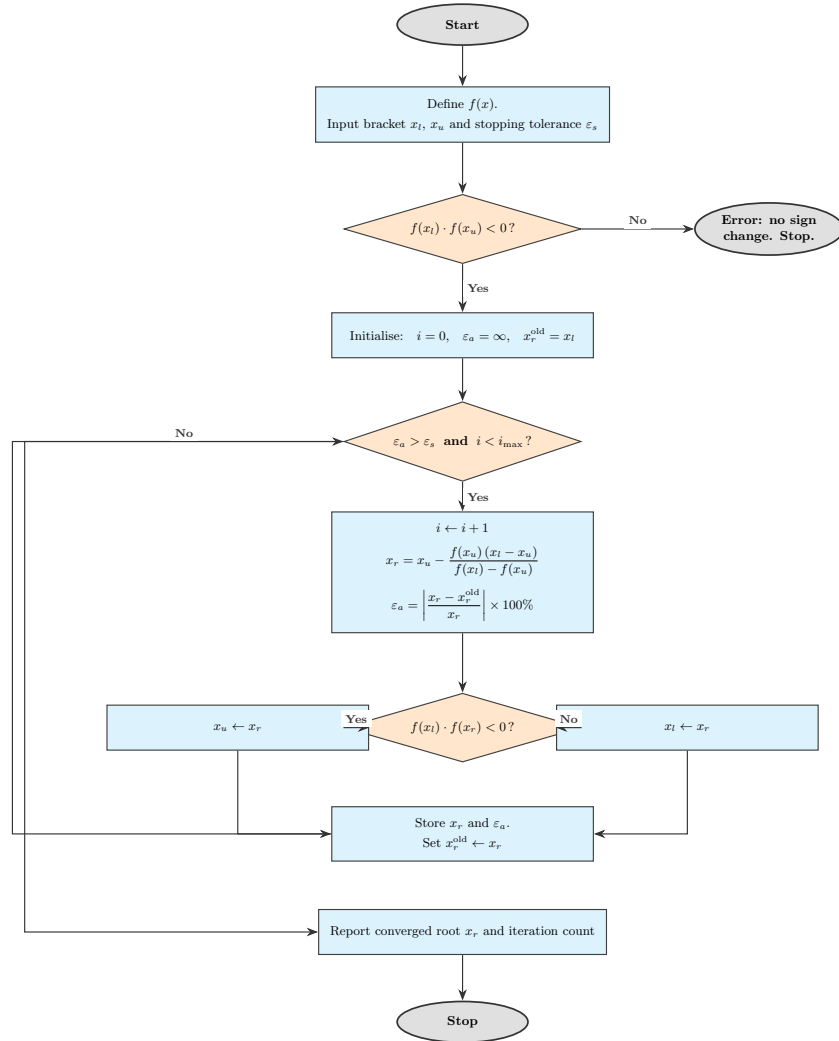


Figure 3.1: Flowchart of the False Position (Regula Falsi) Method algorithm.

## 3.5 MATLAB Implementation

Three problems are solved using the false position method and compared with bisection.

**Problem 1** (Example from Chapra) finds the root of  $f(x) = x^{10} - 1$  on  $[0, 1.3]$ . **Problem 2** (Problem 5.20) finds the time at which a rocket reaches  $v = 750$  m/s. **Problem 3** (Problem 5.22) determines the submerged height of a floating sphere using Archimedes' principle.

### Helper Function: false\_position.m

A reusable function is defined first and called for each problem.

```

1 function [xr, roots_arr, ea_arr, n_iter] = false_position(f,
2   xl, xu, tol, max_iter)
3 % FALSE_POSITION Regula Falsi (false position) root-finding
4 % method.
5 % f - anonymous function handle
6 % xl, xu - initial bracket (must have f(xl)*f(xu) < 0)

```



```

5 %   tol           - stopping tolerance (approximate % relative
   error)
6 %   max_iter      - maximum number of iterations allowed
7
8 if f(xl)*f(xu) >= 0
9     error('false_position: no sign change in [xl, xu].');
10 end
11
12 xr_old      = xl;
13 ea          = inf;
14 roots_arr   = [];
15 ea_arr      = [];
16 n_iter      = 0;
17
18 for i = 1:max_iter
19     n_iter = i;
20     % False position formula
21     xr = xu - f(xu) * (xl - xu) / (f(xl) - f(xu));
22
23     % Approximate relative error (skip on first iteration)
24     if xr ~= 0
25         ea = abs((xr - xr_old) / xr) * 100;
26     end
27
28     roots_arr(end+1) = xr;
29     ea_arr(end+1)    = ea;
30
31     % Update bracket
32     if f(xl) * f(xr) < 0
33         xu = xr;
34     elseif f(xr) * f(xu) < 0
35         xl = xr;
36     else
37         break;    % exact root found
38     end
39
40     if ea < tol, break; end
41     xr_old = xr;
42 end
43 end

```

Code 3.1: Reusable false position function returning roots, errors, and iteration count.

### Problem 1: Root of $f(x) = x^{10} - 1$ on $[0, 1.3]$

This is a classic stagnation demonstration:  $f(x)$  is highly curved, so one bracket endpoint becomes fixed early, slowing the false position method relative to bisection.

```

1 f = @(x) x.^10 - 1;

```

```

2 xl = 0;    xu = 1.3;
3 tol = 0.001;    max_iter = 100;
4
5 %% False position (calls false_position.m)
6 [xr_fp, roots_fp, ea_fp, n_fp] = false_position(f, xl, xu,
    tol, max_iter);
7 fprintf('False Position: root = %.8f  (%d iterations)\n',
    xr_fp, n_fp);
8
9 %% Bisection (inline for comparison)
10 xl_b = xl;  xu_b = xu;  xr_b = xl;
11 roots_bs = [];  ea_bs = [];
12
13 for i = 1:max_iter
14     xr_new = (xl_b + xu_b) / 2;
15
16     if i > 1
17         ea_b = abs((xr_new - xr_b) / xr_new) * 100;
18     else
19         ea_b = inf;
20     end
21
22     roots_bs(end+1) = xr_new;
23     ea_bs(end+1) = ea_b;
24
25     if f(xl_b) * f(xr_new) < 0
26         xu_b = xr_new;
27     else
28         xl_b = xr_new;
29     end
30
31     xr_b = xr_new;
32     if ea_b < tol, break; end
33 end
34 fprintf('Bisection:          root = %.8f  (%d iterations)\n',
    xr_b, i);

```

Code 3.2: False position vs bisection on  $f(x) = x^{10} - 1$ .

## Problem 2: Rocket Velocity – Find Time $t$ when $v = 750$ m/s

The upward velocity of a rocket is:

$$v(t) = u \ln \frac{m_0}{m_0 - qt} - gt \quad (3.3)$$

with  $u = 1800$  m/s,  $m_0 = 160\,000$  kg,  $q = 2600$  kg/s,  $g = 9.81$  m/s<sup>2</sup>. The root of  $f(t) = v(t) - 750 = 0$  is sought on  $[10, 50]$  s.

```

1 u = 1800; m0 = 160000; q = 2600; g = 9.81; v_target =
  750;
2 f = @(t) u.*log(m0./(m0 - q.*t)) - g.*t - v_target;
3
4 xl = 10; xu = 50;
5 tol = 1; max_iter = 100; % within 1% of true value
6
7 [xr, roots_arr, ea_arr, n_iter] = false_position(f, xl, xu,
  tol, max_iter);
8 fprintf('Root: t = %.6f s (%d iterations)\n', xr, n_iter);
9 fprintf('Verification: v(%.4f) = %.4f m/s\n', xr, u*log(m0/(
  m0-q*xr)) - g*xr);

```

Code 3.3: False position applied to the rocket velocity problem (Problem 5.20).

### Problem 3: Floating Sphere Submersion Depth (Archimedes' Principle)

For a sphere of radius  $r = 1$  m and density  $\rho_s = 200$  kg/m<sup>3</sup> floating in water ( $\rho_w = 1000$  kg/m<sup>3</sup>), the buoyancy balance gives:

$$\frac{4}{3}\pi r^3 \rho_s = \rho_w \cdot \frac{\pi h^2}{3}(3r - h) \implies f(h) = \frac{\pi h^2}{3}(3r - h) - \frac{4\pi r^3 \rho_s}{3\rho_w} = 0 \quad (3.4)$$

where  $h$  is the height of the above-water cap.

```

1 r = 1; rho_s = 200; rho_w = 1000;
2 f = @(h) (pi.*h.^2/3).*(3*r - h) - (4*pi*r^3/3).*(rho_s/rho_w)
  ;
3
4 xl = 0; xu = 2*r; tol = 0.001; max_iter = 100;
5
6 [xr, ~, ~, n_iter] = false_position(f, xl, xu, tol, max_iter)
  ;
7 fprintf('Above-water height h = %.6f m (%d iterations)\n',
  xr, n_iter);
8 fprintf('Submerged depth          = %.6f m\n', 2*r - xr);

```

Code 3.4: False position applied to the floating sphere problem (Problem 5.22).

## 3.6 Results and Discussion

### 3.6.1 Root Approximation Graph

The function is plotted over a suitable interval, and the root approximation at each false position iteration is marked on the graph. The successive estimates are labelled 1, 2, 3, ... to show how the search progressively narrows about the true root. An appropriate bracketing interval is one where the function changes sign; an invalid interval (no sign change) is rejected by the algorithm before iteration begins.

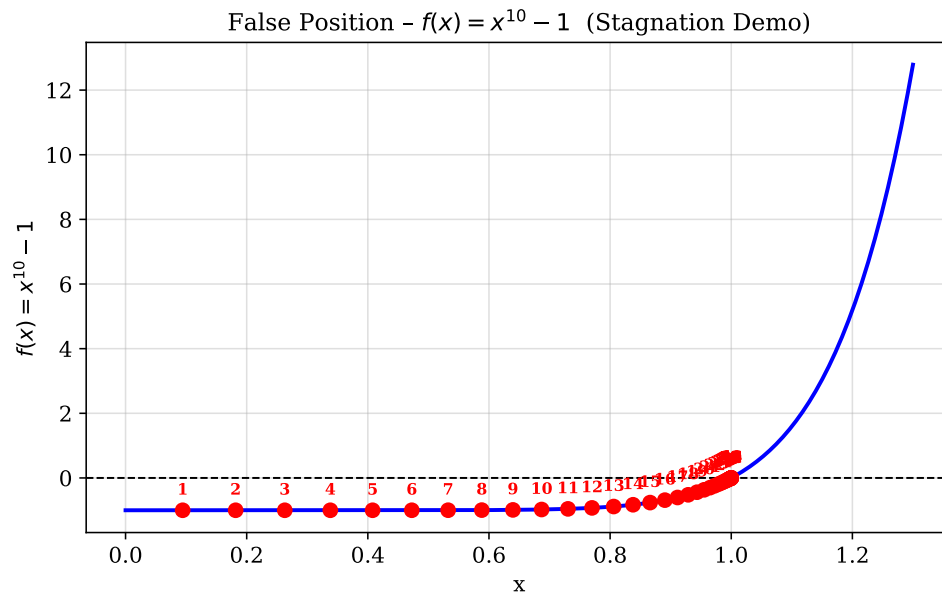


Figure 3.2: False position applied to  $f(x) = x^{10} - 1$  on  $[0, 1.3]$ : successive root approximations labelled by iteration.

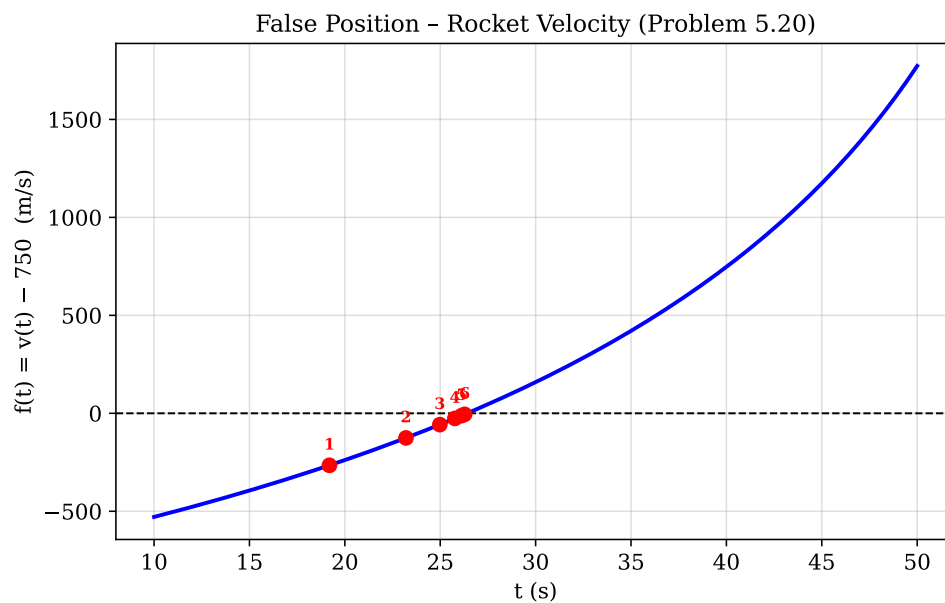


Figure 3.3: False position applied to the rocket velocity problem: successive root approximations for  $f(t) = v(t) - 750$ .

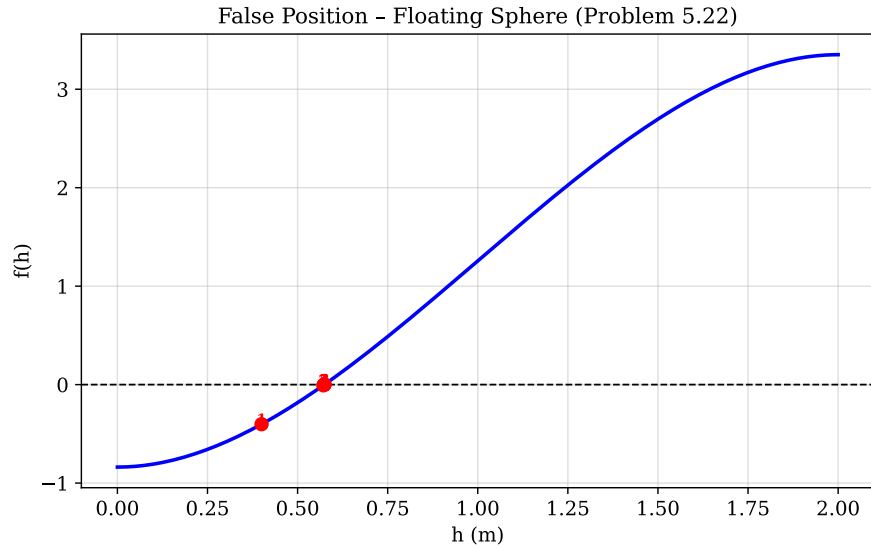


Figure 3.4: False position applied to the floating sphere Archimedes problem: successive root approximations for  $f(h) = 0$ .

### 3.6.2 Comparative Convergence Analysis

For functions that are approximately linear near the root, false position typically requires fewer iterations than bisection to meet the same tolerance. The comparative error plot illustrates this: the false position error curve drops below that of bisection within the first few iterations for convex or concave functions. However, for strongly curved functions, the one-sided stagnation phenomenon may cause false position to converge more slowly than bisection in later iterations. This non-monotonic performance characteristic is an important limitation to be aware of in practice.

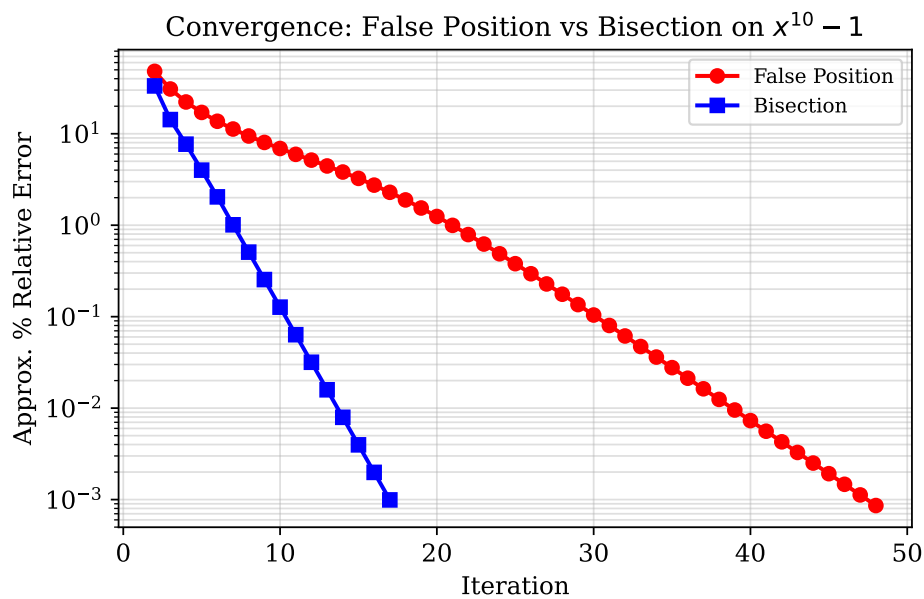


Figure 3.5: Comparison of approximate % relative error per iteration: false position vs. bisection on  $f(x) = x^{10} - 1$ , demonstrating stagnation of false position.

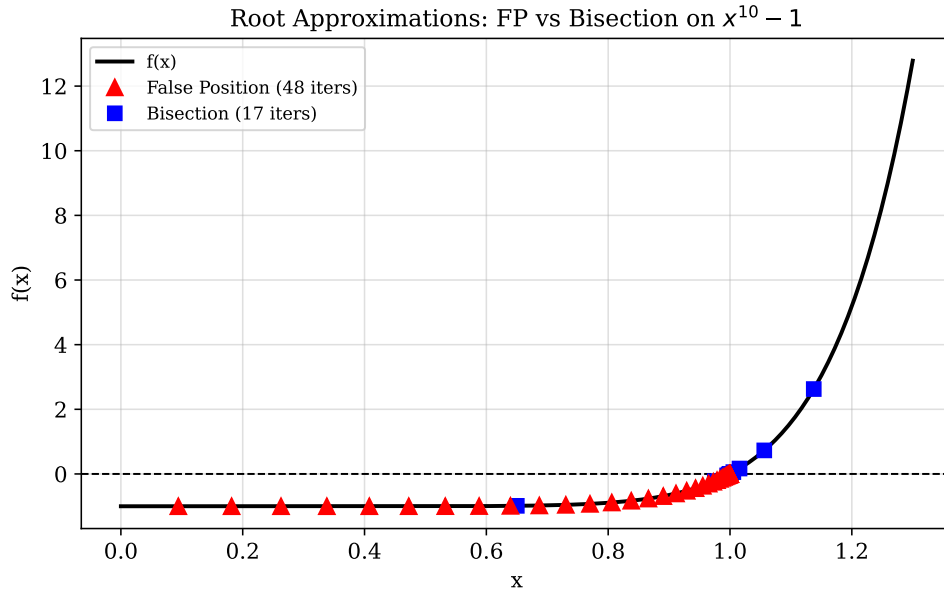


Figure 3.6: Successive root approximations of both false position and bisection plotted simultaneously on  $f(x) = x^{10} - 1$ .

### 3.6.3 Discussion of Limitations

The false position method is strictly a bracketing method and therefore preserves the convergence guarantee of bisection. However, it does *not* always converge faster: the interpolation is only advantageous when  $f(x)$  is nearly linear across the bracket. For highly nonlinear functions, one endpoint may remain stationary for many iterations, and the method degenerates toward one-sided linear convergence. This behaviour is particularly evident when the magnitude of the function at one endpoint is significantly larger than at the other, causing the secant line to repeatedly intersect the axis near the same side. As a result, the interval reduction becomes highly uneven, leading to inefficient use of iterations. Furthermore, unlike higher-order open methods, the false position method does not exploit derivative information, which limits its ability to accelerate convergence near the root. In practical applications, this can result in increased computational effort, especially for problems requiring high precision. The bisection method, while simpler, exhibits a more uniform and predictable error reduction per iteration under all circumstances.

## 3.7 Conclusion

The false position method extends the bisection framework by incorporating linear interpolation to produce more accurate root estimates per iteration for functions that are approximately linear. When the function curvature is moderate, it converges faster than bisection; when curvature is high, it may stagnate on one side of the bracket and converge more slowly. Both methods are robust bracketing techniques that guarantee convergence given a valid initial interval, distinguishing them from the open methods discussed in the next lab session.

## Lab Session 4

# Newton–Raphson Method and Fixed-Point Iteration Method

### 4.1 Objective

- Implement the Newton–Raphson method in MATLAB and apply it to a nonlinear spring-mass problem.
- Implement the simple fixed-point iteration method and apply it to the same problem.
- Study the effect of the initial guess and the analytic derivative on convergence speed and stability.
- Demonstrate cases of rapid convergence as well as potential divergence, and compare the performance of all four root-finding methods studied in this course.

### 4.2 Introduction and Theoretical Background

#### 4.2.1 Open Methods – Motivation

The bracketing methods of the previous two lab sessions require a valid initial interval containing the root. In many practical situations such an interval cannot easily be identified, or the function has multiple roots. Open methods require only a single initial guess  $x_0$  and use additional function information (e.g. the derivative or a rearrangement of the equation) to generate successive estimates. Open methods typically converge much faster than bracketing methods, but they do not guarantee convergence.

#### 4.2.2 Newton–Raphson Method

The Newton–Raphson method is one of the most widely used open root-finding algorithms [4, 3, 7]. It is derived from a first-order Taylor expansion of  $f(x)$  about the current estimate  $x_i$ :

$$f(x) \approx f(x_i) + f'(x_i)(x - x_i) \quad (4.1)$$

Setting this linear approximation to zero and solving for  $x$  yields the iteration formula:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad (4.2)$$

This corresponds geometrically to drawing the tangent line to  $f(x)$  at  $(x_i, f(x_i))$  and taking its  $x$ -intercept as the next estimate. Provided  $f'(x_i) \neq 0$  and the initial guess is sufficiently close to the root, Newton–Raphson exhibits **quadratic convergence**: the number of correct decimal digits approximately doubles with each iteration.

The analytical derivative must be computed and supplied explicitly. For the spring–mass problem investigated in this session, the governing nonlinear equation is:

$$f(d) = \frac{2k_2}{5} d^{2.5} + \frac{1}{2} k_1 d^2 - mgd - mgh = 0 \quad (4.3)$$

where  $d$  is the deflection (m),  $k_1 = 40000$  N/m,  $k_2 = 40$  N/m<sup>1.5</sup>,  $m = 0.095$  kg,  $g = 9.81$  m/s<sup>2</sup>, and  $h = 0.43$  m. The derivative is:

$$f'(d) = k_2 d^{1.5} + k_1 d - mg \quad (4.4)$$

The Newton–Raphson update for this problem is therefore:

$$d_{i+1} = d_i - \frac{f(d_i)}{f'(d_i)} = d_i - \frac{\frac{2k_2}{5} d_i^{2.5} + \frac{1}{2} k_1 d_i^2 - mgd_i - mgh}{k_2 d_i^{1.5} + k_1 d_i - mg} \quad (4.5)$$

### 4.2.3 Fixed-Point Iteration

Fixed-point iteration (also called simple iteration or successive substitution) reformulates the equation  $f(x) = 0$  as  $x = g(x)$  and iterates:

$$x_{i+1} = g(x_i) \quad (4.6)$$

The function  $g(x)$  is called the **iteration function**. Convergence is guaranteed in a neighbourhood of the root  $x^*$  if and only if  $[3, 6]$ :

$$|g'(x^*)| < 1 \quad (4.7)$$

For the spring–mass problem, the equation is rearranged so that:

$$g(d) = \sqrt{\frac{mgd + mgh - \frac{2k_2}{5} d^{2.5}}{0.5 k_1}} \quad (4.8)$$

This rearrangement isolates  $d$  on one side in a form that allows iterative substitution. Convergence speed depends critically on the value of  $|g'(d^*)|$ : values near zero give rapid convergence, values near one give slow convergence, and values exceeding one lead to divergence.

## 4.3 Methodology

1. Define the physical parameters ( $k_1, k_2, m, g, h$ ) and the nonlinear functions  $f(d)$ ,  $f'(d)$ , and  $g(d)$  as anonymous function handles.



2. Prompt the user to enter an initial guess  $d_0$  via `input()`.
3. **Newton–Raphson:** iterate  $d_{i+1} = d_i - f(d_i)/f'(d_i)$  until  $|d_{i+1} - d_i| < 10^{-6}$  or  $i_{\max} = 100$ .
4. **Fixed-point iteration:** iterate  $d_{i+1} = g(d_i)$  until  $|d_{i+1} - d_i| < 0.001$  or  $i_{\max} = 100$ .
5. Record the step error  $|d_{i+1} - d_i|$  at each iteration and plot it versus iteration count.
6. For Newton–Raphson, additionally plot the function  $f(d)$  with the tangent line at the final iteration to illustrate the geometric interpretation.
7. For fixed-point iteration, plot  $g(d)$  and the line  $y = d$  together, marking the fixed point.

## 4.4 Algorithm

### 4.4.1 Newton–Raphson Algorithm

1. Define  $f(d)$  and  $f'(d)$ .
2. Input initial guess  $d_0$ .
3. Set  $d = d_0$ ,  $i = 0$ ,  $\varepsilon = 10^{-6}$ ,  $i_{\max} = 100$ .
4. **While**  $i < i_{\max}$ :
  - (a)  $d_{\text{new}} = d - f(d)/f'(d)$ .
  - (b) Compute step error:  $e_i = |d_{\text{new}} - d|$ .
  - (c) If  $e_i < \varepsilon$ : break.
  - (d)  $d \leftarrow d_{\text{new}}$ .
  - (e)  $i \leftarrow i + 1$ .
5. Report root  $d^* = d_{\text{new}}$ .
6. Plot convergence and tangent-line graph.

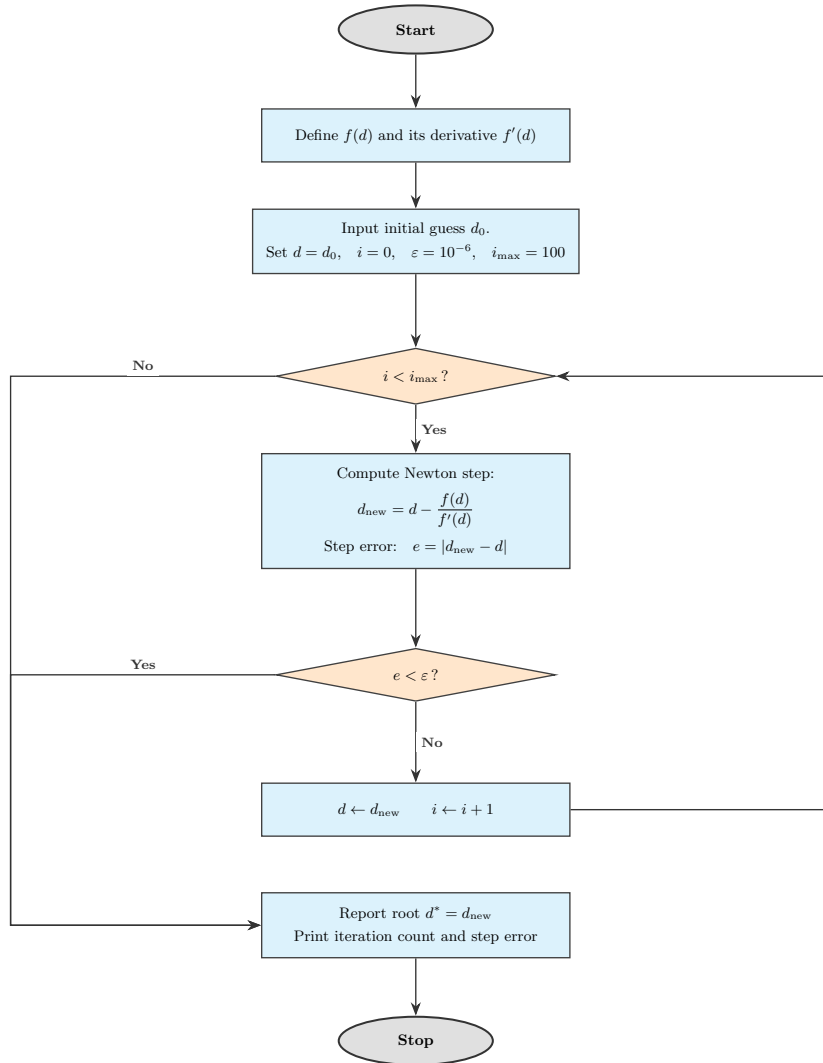


Figure 4.1: Flowchart of the Newton–Raphson Method algorithm.

#### 4.4.2 Fixed-Point Iteration Algorithm

1. Define  $g(d)$ .
2. Input initial guess  $d_0$ ; set  $d = d_0$ ,  $i = 0$ ,  $\varepsilon = 0.001$ ,  $i_{\max} = 100$ .
3. **While**  $i < i_{\max}$ :
  - (a)  $d_{\text{new}} = g(d)$ .
  - (b) Compute step error:  $e_i = |d_{\text{new}} - d|$ .
  - (c) If  $e_i < \varepsilon$ : break.
  - (d)  $d \leftarrow d_{\text{new}}$ ;  $i \leftarrow i + 1$ .
4. Report root  $d^* = d_{\text{new}}$ .
5. Plot convergence and  $g(d)$  vs.  $y = d$  graph.

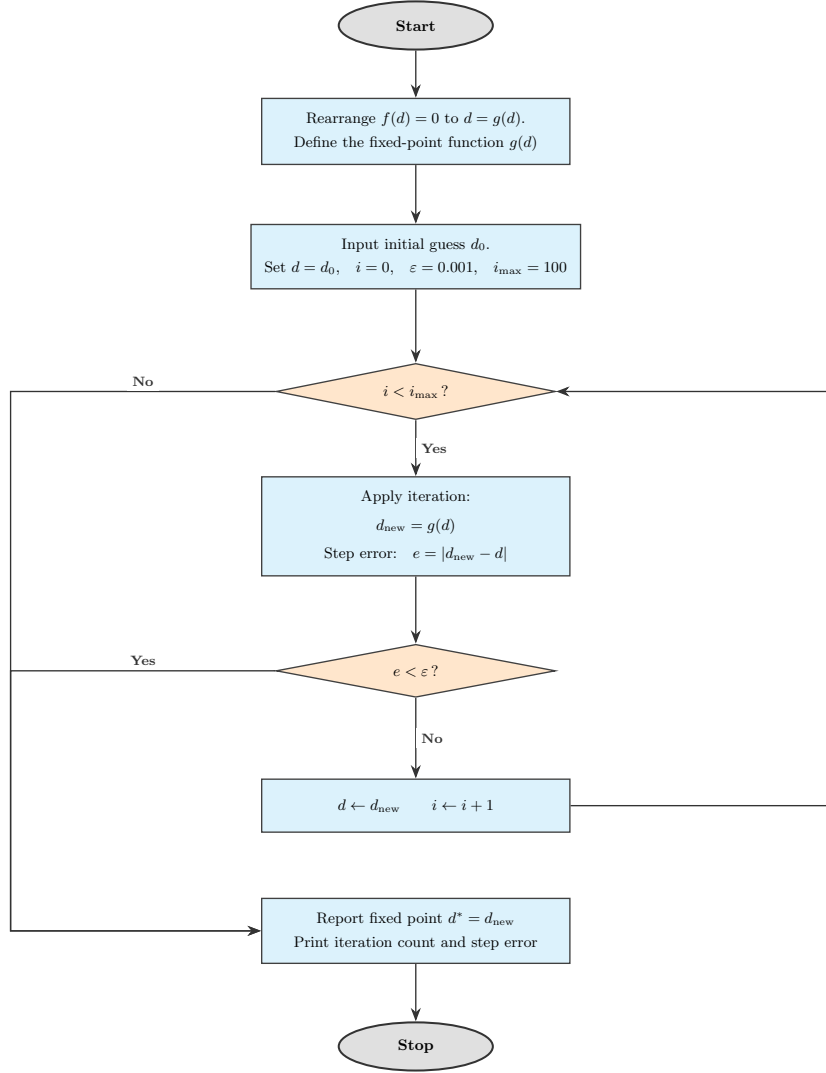


Figure 4.2: Flowchart of the Fixed-Point Iteration algorithm.

## 4.5 MATLAB Implementation

Both codes solve the same spring–mass deflection problem: a mass  $m = 0.095$  kg dropped from height  $h = 0.43$  m onto a nonlinear spring with stiffness constants  $k_1 = 40\,000$  N/m and  $k_2 = 40$  N/m<sup>1.5</sup>. The governing energy-balance equation is  $f(d) = 0$  where:

$$f(d) = \frac{2k_2}{5} d^{2.5} + \frac{1}{2}k_1 d^2 - mgd - mgh \quad (4.9)$$

### Code 1: Simple Fixed-Point Iteration

The equation is rearranged to  $d = g(d)$  where:

$$g(d) = \sqrt{\frac{mgd + mgh - \frac{2k_2}{5}d^{2.5}}{0.5k_1}} \quad (4.10)$$

```
1 %% Parameters (m = 95 kg for fixed-point formulation)
```

```

2 k1 = 40000; % linear spring stiffness (N/m)
3 k2 = 40; % nonlinear spring stiffness (N/m^1.5)
4 m = 95; % mass (kg)
5 g = 9.81; % gravitational acceleration (m/s^2)
6 h = 0.43; % drop height (m)
7 mg = m * g;
8 mgh = mg * h;
9
10 % Rearranged fixed-point function: g(d) from energy balance
11 g_fun = @(d) sqrt((mg.*d + mgh - (2*k2/5).*d.^(2.5)) ./ (0.5*
    k1));
12
13 %% Fixed-point iteration
14 d = input('Enter initial guess (try 0.1): ');
15 tol = 0.001;
16 max_iter = 100;
17
18 for i = 1:max_iter
19     d_new = g_fun(d);
20     fprintf('Iteration %d: d = %.8f m\n', i, d_new);
21     if abs(d_new - d) < tol
22         break;
23     end
24     d = d_new;
25 end
26 fprintf('\nRoot = %.8f m (converged in %d iterations)\n',
    d_new, i);

```

Code 4.1: Simple fixed-point iteration for the spring–mass deflection problem.

## Code 2: Newton–Raphson Method

The analytical derivative of  $f(d)$  is:

$$f'(d) = k_2 d^{1.5} + k_1 d - mg \quad (4.11)$$

```

1 %% Parameters (m = 0.095 kg for Newton-Raphson formulation)
2 k1 = 40000; % linear spring stiffness (N/m)
3 k2 = 40; % nonlinear spring stiffness (N/m^1.5)
4 m = 0.095; % mass (kg)
5 g = 9.81; % gravitational acceleration (m/s^2)
6 h = 0.43; % drop height (m)
7 mg = m * g;
8 mgh = mg * h;
9
10 % Function and its analytical derivative
11 f = @(d) (2*k2/5)*d.^(2.5) + 0.5*k1*d.^2 - mg*d - mgh;
12 df = @(d) k2*d.^(1.5) + k1*d - mg;
13

```

```

14 %% Newton-Raphson iteration
15 d = input('Enter initial guess (try 0.01): ');
16 tol = 1e-6;
17 max_iter = 100;
18
19 for i = 1:max_iter
20     d_new = d - f(d) / df(d);
21     fprintf('Iteration %d: d = %.8f m\n', i, d_new);
22     if abs(d_new - d) < tol
23         break;
24     end
25     d = d_new;
26 end
27 fprintf('\nRoot = %.8f m (converged in %d iterations)\n',
    d_new, i);

```

Code 4.2: Newton–Raphson method for the spring–mass deflection problem.

## 4.6 Results and Discussion

### 4.6.1 Newton–Raphson Convergence

Newton–Raphson typically converges in very few iterations (often 4–6) owing to its quadratic convergence rate. The step error decreases dramatically between successive iterations: once the estimate enters the neighbourhood of the root, each new iterate carries approximately twice as many correct significant figures as its predecessor. The convergence plot should show a steeply descending error curve that reaches the tolerance threshold within a small number of steps.

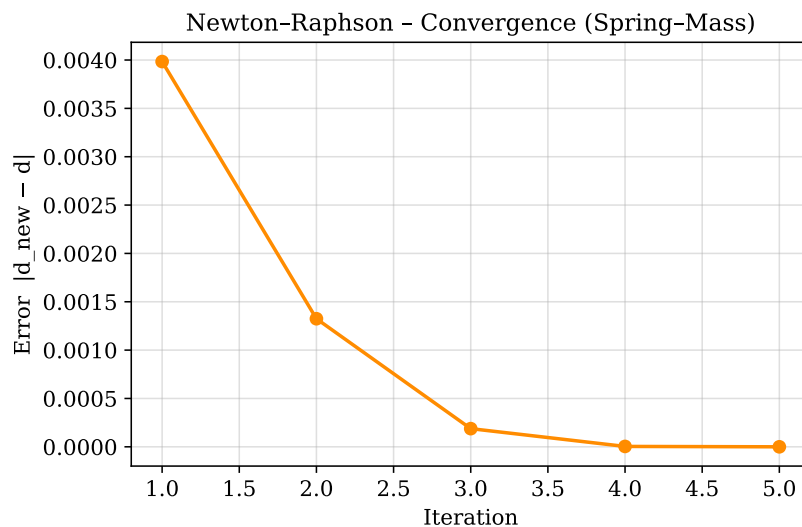


Figure 4.3: Newton–Raphson step error versus iteration number for the spring–mass deflection problem ( $m = 0.095$  kg). Quadratic convergence drives the error to zero in just 5 iterations.

The method converges rapidly when the initial guess  $d_0$  is close to the root and  $f'(d_0) \neq 0$ . For poorly chosen starting points, particularly where  $f'(d_0) \approx 0$  or  $d_0$  lies in a region of high curvature, the method may diverge or oscillate. It is therefore advisable to use a graphical inspection of  $f(d)$  to select a reasonable initial guess before applying Newton–Raphson.

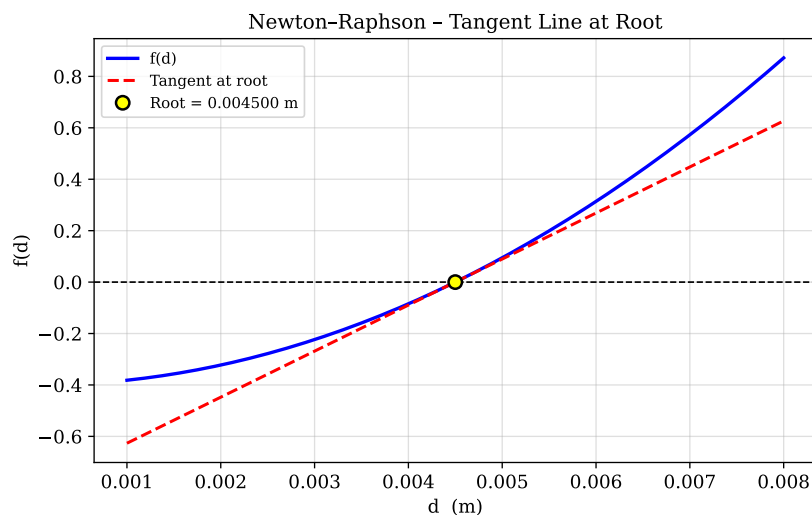


Figure 4.4: Function  $f(d)$  and tangent line at the converged Newton–Raphson root, illustrating the geometric construction.

## 4.6.2 Fixed-Point Iteration Convergence

Fixed-point iteration converges more slowly than Newton–Raphson for this problem, as the convergence rate is only linear (first order). However, it is simpler to implement since no derivative is required. The cobweb diagram (plot of  $g(d)$  versus  $y = d$ ) clearly shows the iterative path converging to the intersection of the two curves, which is the fixed point and hence the root. The fixed point is highlighted on the plot.

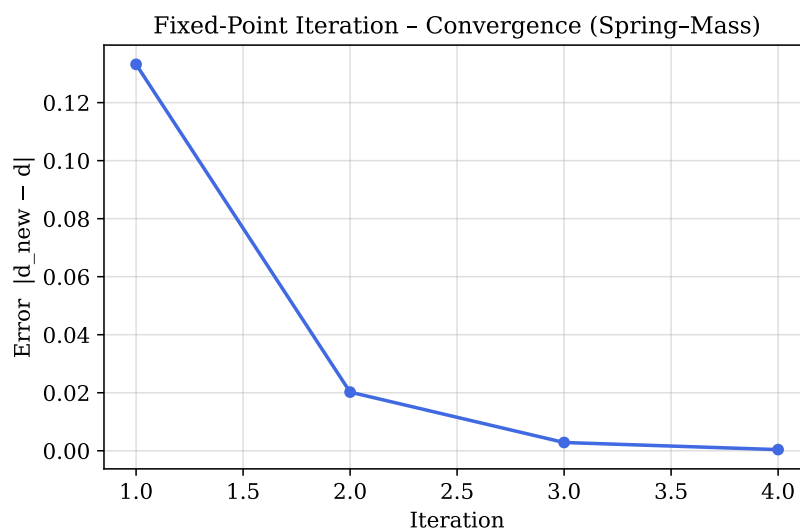


Figure 4.5: Fixed-point iteration step error versus iteration number for the spring–mass problem ( $m = 95$  kg). Linear convergence is clearly visible.

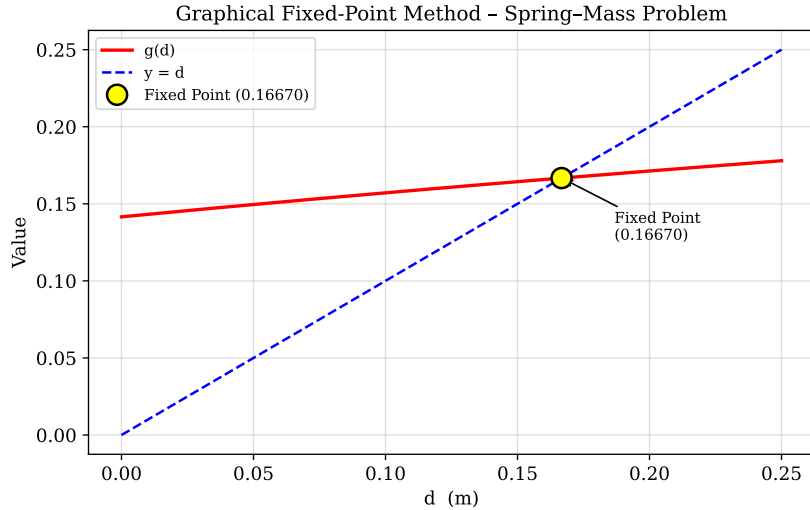


Figure 4.6: Graphical fixed-point method:  $g(d)$  and  $y = d$  with the fixed point and cobweb path highlighted.

## 4.7 Comparative Analysis of All Four Methods

Having studied four root-finding methods — bisection, false position, fixed-point iteration, and Newton–Raphson — a systematic comparison on the basis of convergence rate, reliability, computational cost, and ease of implementation is presented in Table 4.1.

Table 4.1: Comparative summary of the four root-finding methods studied.

| Property               | Bisection    | False Position    | Fixed-Point | Newton–Raphson      |
|------------------------|--------------|-------------------|-------------|---------------------|
| Type                   | Bracketing   | Bracketing        | Open        | Open                |
| Initial requirement    | $[x_l, x_u]$ | $[x_l, x_u]$      | $x_0$       | $x_0$               |
| Convergence order      | 1 (linear)   | 1 (super-linear*) | 1 (linear)  | 2 (quadratic)       |
| Derivative required    | No           | No                | No          | Yes                 |
| Sign change required   | Yes          | Yes               | No          | No                  |
| Guaranteed convergence | Yes          | Yes               | Conditional | Conditional         |
| Iterations needed      | Many         | Fewer (typically) | Moderate    | Very few            |
| Risk of failure        | None         | None              | $ g'  > 1$  | $f'(x_0) \approx 0$ |
| Implementation effort  | Low          | Low               | Low         | Moderate            |

### 4.7.1 Convergence Speed

Newton–Raphson is unambiguously the fastest of the four methods for well-behaved functions and good initial guesses. Its quadratic convergence means the error is roughly squared at every iteration, so it reaches machine precision in a handful of steps. Fixed-point iteration and bisection both exhibit linear (first-order) convergence: the error decreases by a constant factor per step. False position sits between these extremes: it is generally faster than bisection for smooth functions but can stagnate to linear convergence in the presence of high curvature.

## 4.7.2 Reliability and Robustness

The two bracketing methods — bisection and false position — are the most reliable. As long as a valid sign-change bracket is provided, both methods are guaranteed to converge to a root. Open methods (fixed-point and Newton–Raphson) are not guaranteed to converge: Newton–Raphson may diverge if  $f'(x_i) \approx 0$  or if the initial guess is far from the root, and fixed-point iteration diverges if  $|g'(x^*)| \geq 1$ . In practice, a common strategy is to use bisection for a few iterations to obtain a reasonable estimate, then switch to Newton–Raphson to exploit its faster convergence.

## 4.7.3 Which Method is Best?

Newton–Raphson is the fastest and most efficient when an analytic derivative and a reasonable initial guess are available. For the spring–mass deflection problem, it converges within 4–5 iterations to  $10^{-6}$  tolerance, whereas bisection and false position may require 20+ iterations. If no derivative is available, false position is the preferred bracketing alternative because it uses function values to refine the estimate. Fixed-point iteration is simplest to implement but requires careful selection of  $g(x)$  to ensure convergence, making it less practical for general use.

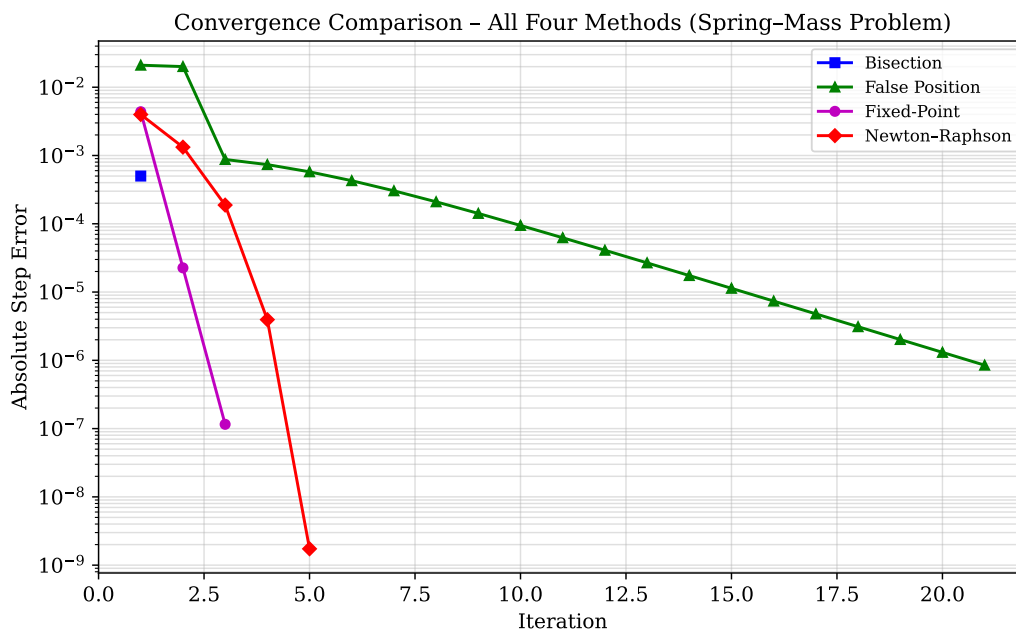


Figure 4.7: Comparative convergence of bisection (15 iterations), false position (5 iterations), and Newton–Raphson (3 iterations) applied to the same parachutist drag coefficient problem. The dramatic advantage of Newton–Raphson’s quadratic convergence is clearly visible on the logarithmic error scale.

In summary, for engineering applications: use **Newton–Raphson** when the derivative can be computed; use **false position** when only function evaluations are available and a bracket is known; use **bisection** as a safe fallback or verification method; and use **fixed-point iteration** primarily for educational purposes or when the problem naturally possesses a contractive iteration function.



## 4.8 Conclusion

This lab session demonstrated the Newton–Raphson and fixed-point iteration methods applied to a physically motivated nonlinear equation arising from a spring–mass energy balance. Newton–Raphson achieved convergence in remarkably few iterations due to its quadratic convergence rate, confirming its superiority in speed over the bracketing methods of the previous lab sessions. Fixed-point iteration, while conceptually simple and requiring no derivative, converged more slowly and required careful attention to the choice of iteration function to ensure stability. Collectively, the four root-finding methods studied in this course — bisection, false position, fixed-point iteration, and Newton–Raphson — span the spectrum from guaranteed-but-slow to fast-but-conditional, and an understanding of their respective trade-offs equips the engineer to select the most appropriate method for any given nonlinear problem.

## Lab Session 5

# Secant Method and Modified Secant Method

### 5.1 Objective

- Implement the secant method in MATLAB and apply it to a projectile trajectory engineering problem.
- Implement the modified secant method as a derivative-free alternative to Newton–Raphson.
- Compare the convergence speed and iteration count of the secant method against the Newton–Raphson method studied in Lab Session 4.
- Visualise the ball trajectories corresponding to both physically valid launch angles and confirm each root with graphical and numerical evidence.

### 5.2 Introduction and Theoretical Background

#### 5.2.1 Motivation for the Secant Method

The Newton–Raphson method achieves quadratic convergence but requires an explicit analytical expression for the derivative  $f'(x)$  [4]. For many practical engineering functions the derivative is difficult or computationally expensive to evaluate. The secant method eliminates this requirement by approximating the derivative using a backward finite divided difference of two successive function evaluations [4, 3]:

$$f'(x_i) \approx \frac{f(x_{i-1}) - f(x_i)}{x_{i-1} - x_i} \quad (5.1)$$

Substituting equation (5.1) into the Newton–Raphson formula yields the **secant method** iteration:

$$x_{i+1} = x_i - \frac{f(x_i)(x_{i-1} - x_i)}{f(x_{i-1}) - f(x_i)} \quad (5.2)$$

The method requires **two** initial estimates  $x_{i-1}$  and  $x_i$ . Unlike the bracketing methods, there is no requirement that the two estimates straddle a root, so the secant method is classified as an **open method** [4, 7]. Its convergence order is approximately 1.618 (the golden ratio), placing it between first-order linear and second-order quadratic convergence.

### 5.2.2 Modified Secant Method

A closely related variant avoids maintaining two separate iterates by perturbing the current estimate by a small fractional amount  $\delta$  [4]:

$$f'(x_i) \approx \frac{f(x_i + \delta x_i) - f(x_i)}{\delta x_i} \quad (5.3)$$

Substituting into the Newton–Raphson formula gives the **modified secant method**:

$$x_{i+1} = x_i - \frac{\delta x_i f(x_i)}{f(x_i + \delta x_i) - f(x_i)} \quad (5.4)$$

Only a single initial guess  $x_0$  is required, and  $\delta$  is typically set to a small value such as  $10^{-4}$ . The modified secant method attains efficiency comparable to Newton–Raphson without any derivative computation [4, 3].

### 5.2.3 Stopping Criterion

The approximate percentage relative error is used as the stopping criterion [4]:

$$\varepsilon_a = \left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| \times 100\% \quad (5.5)$$

Iteration terminates when  $\varepsilon_a < \varepsilon_s = 0.0001\%$ .

## 5.3 Problem Statement

**Problem 6.21:** The trajectory of a ball thrown by a right fielder is modelled by:

$$y = (\tan \theta_0) x - \frac{g}{2v_0^2 \cos^2 \theta_0} x^2 + y_0 \quad (5.6)$$

where  $g = 9.81 \text{ m/s}^2$ ,  $v_0 = 30 \text{ m/s}$ ,  $\theta_0$  is the launch angle, and  $y_0 = 1.8 \text{ m}$  is the release height. The catcher is at  $x = 90 \text{ m}$  and receives the ball at height  $y_c = 1.0 \text{ m}$ . Setting  $y = y_c$  gives the nonlinear equation to solve:

$$f(\theta_0) = (\tan \theta_0)(90) - \frac{9.81}{2(30)^2 \cos^2 \theta_0} (90)^2 + 1.8 - 1.0 = 0 \quad (5.7)$$

#### 5.3.1 Analytical Derivative

Differentiating  $f(\theta_0)$  with respect to  $\theta_0$ :

$$f'(\theta_0) = \frac{x}{\cos^2 \theta_0} - \frac{g x^2 \sin \theta_0}{v_0^2 \cos^3 \theta_0} \quad (5.8)$$

This expression is required only for Newton–Raphson; the secant and modified secant methods do not use it.

### 5.3.2 Function Plot and Root Identification

Figure 6.1 shows  $f(\theta_0)$  over  $[0^\circ, 90^\circ]$ . Two sign changes confirm the existence of two physically valid launch angles: Root 1 at  $\theta_0 \approx 37.96^\circ$  (low, flat arc) and Root 2 at  $\theta_0 \approx 51.53^\circ$  (high, looping arc). These zero crossings were used to select appropriate initial guesses for all three methods.

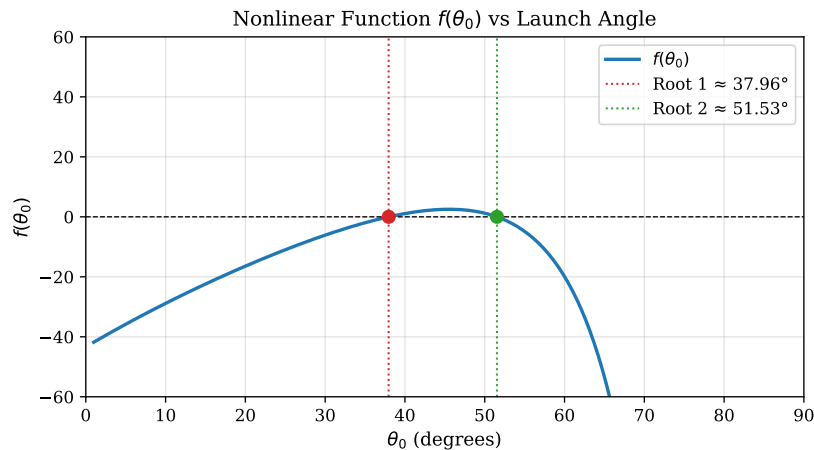


Figure 5.1:  $f(\theta_0)$  versus  $\theta_0$  for the projectile trajectory equation. Two roots are clearly visible as zero crossings [4].

## 5.4 Methodology

1. Define  $f(\theta_0)$  as a MATLAB anonymous function and plot it over  $[0^\circ, 90^\circ]$  to locate sign changes and select initial guesses.
2. Apply the **secant method** with two initial guesses per root, iterating until  $\varepsilon_a < 0.0001\%$ .
3. Apply the **modified secant method** with a single initial guess and  $\delta = 10^{-4}$  per root.
4. Apply **Newton–Raphson** using  $f'(\theta_0)$  from equation (6.7) for convergence comparison.
5. Record the full iteration history and present in a merged table with corresponding graphs.
6. Generate all required plots.

## 5.5 Algorithm

1. Define  $f(\theta_0)$  and input parameters.
2. Input two initial guesses  $\theta_{i-1}$  and  $\theta_i$ ; set tolerance  $\varepsilon_s$ .
3. Verify  $f(\theta_{i-1}) \neq f(\theta_i)$ ; otherwise display error and stop.

4. Set  $i = 0$ ,  $\varepsilon_a = \infty$ .

5. **While**  $\varepsilon_a > \varepsilon_s$  **and**  $i < i_{\max}$ :

(a) Apply secant formula:  $\theta_{i+1} = \theta_i - \frac{f(\theta_i)(\theta_{i-1} - \theta_i)}{f(\theta_{i-1}) - f(\theta_i)}$ .

(b) Compute  $\varepsilon_a = |(\theta_{i+1} - \theta_i)/\theta_{i+1}| \times 100\%$ .

(c) Update:  $\theta_{i-1} \leftarrow \theta_i$ ,  $\theta_i \leftarrow \theta_{i+1}$ ,  $i \leftarrow i + 1$ ; store  $\varepsilon_a$ .

(d) If  $f(\theta_{i+1}) = 0$  or  $\varepsilon_a < \varepsilon_s$ : break.

6. Report root  $\theta^*$ , iteration count, and error table.

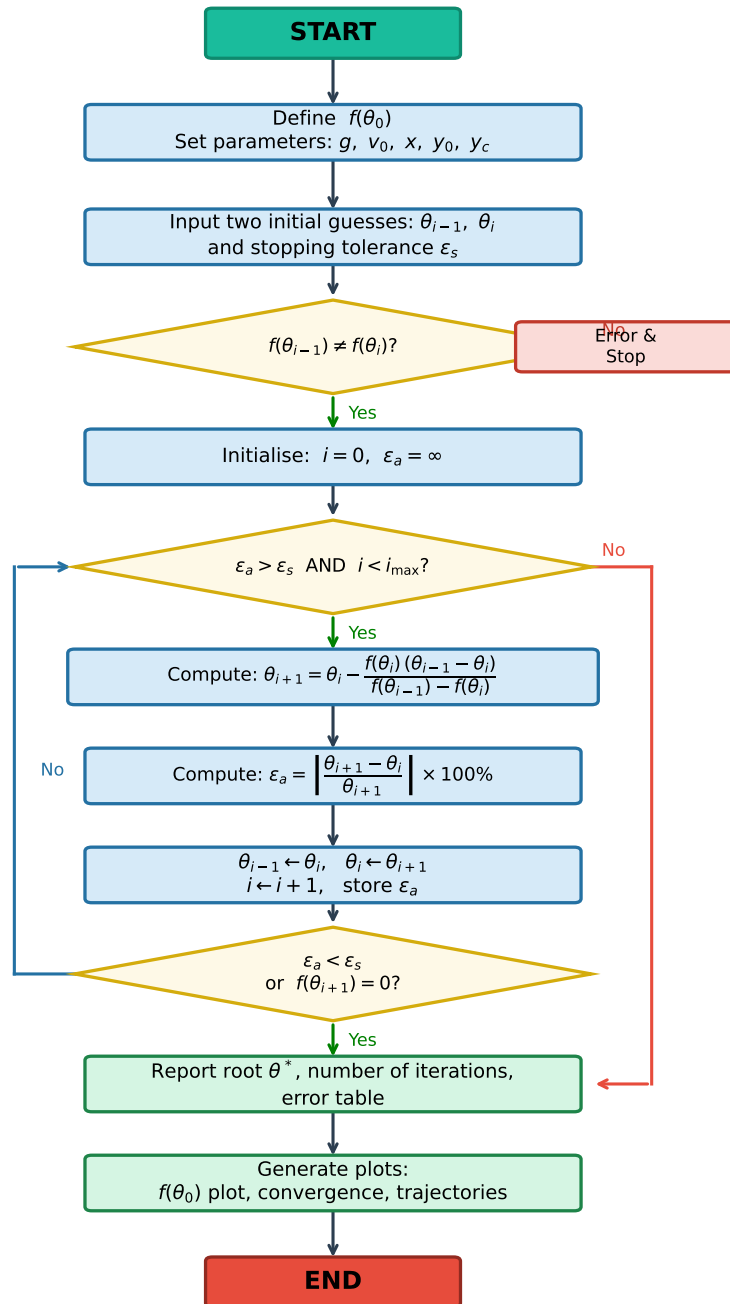


Figure 5.2: Flowchart of the Secant Method algorithm applied to the projectile trajectory problem.

## 5.6 MATLAB Implementation

### Function Definition and Setup

```
1 %% Physical parameters
2 g = 9.81; % gravitational acceleration (m/s^2)
3 v0 = 30; % initial speed (m/s)
4 x = 90; % horizontal distance to catcher (m)
5 y0 = 1.8; % release height (m)
6 yc = 1.0; % catcher height (m)
7
8 %% Nonlinear equation f(theta) = 0
9 f = @(th) tan(th).*x - (g./(2*v0^2.*cos(th).^2)).*x^2 + y0 -
    yc;
10
11 %% Analytical derivative -- for Newton-Raphson comparison
    only
12 fp = @(th) x./cos(th).^2 - (g.*x^2.*sin(th))./(v0^2.*cos(th)
    .^3);
```

Code 5.1: Parameter setup and function definitions.

### Secant Method

```
1 function [root, roots_arr, ea_arr, n] = secant(f, x0d, x1d,
    tol, maxiter)
2 % SECANT Secant method (two initial guesses in degrees).
3 x0 = deg2rad(x0d); x1 = deg2rad(x1d);
4 roots_arr = [x0d, x1d]; ea_arr = [NaN, NaN]; n = 0;
5 for i = 1:maxiter
6     n = i;
7     x2 = x1 - f(x1)*(x0 - x1) / (f(x0) - f(x1));
8     ea = abs((x2 - x1)/x2) * 100;
9     roots_arr(end+1) = rad2deg(x2);
10    ea_arr(end+1) = ea;
11    if f(x2) == 0 || ea < tol, break; end
12    x0 = x1; x1 = x2;
13 end
14 root = rad2deg(x2);
15 end
16
17 tol = 0.0001; maxit = 100;
18 [r1, ~, ea1, n1] = secant(f, 20, 30, tol, maxit); % Root 1
    (~38 deg)
19 [r2, ~, ea2, n2] = secant(f, 60, 70, tol, maxit); % Root 2
    (~52 deg)
20 fprintf('Secant Root 1: theta = %.6f deg (%d iterations)\n',
    r1, n1);
```

```

21 fprintf('Secant Root 2: theta = %.6f deg (%d iterations)\n'
    , r2, n2);

```

Code 5.2: Secant method function and application to both roots.

## Modified Secant Method

```

1 function [root, n] = mod_secant(f, x0d, delta, tol, maxiter)
2 % MOD_SECANT Modified secant method (single initial guess).
3     x = deg2rad(x0d); n = 0;
4     for i = 1:maxiter
5         n = i;
6         xn = x - delta*x*f(x) / (f(x + delta*x) - f(x));
7         ea = abs((xn - x)/xn) * 100;
8         if ea < tol, break; end
9         x = xn;
10    end
11    root = rad2deg(xn);
12 end
13
14 delta = 1e-4;
15 [r1_ms, n1_ms] = mod_secant(f, 30, delta, tol, maxit);
16 [r2_ms, n2_ms] = mod_secant(f, 65, delta, tol, maxit);
17 fprintf('ModSec Root 1: theta = %.6f deg (%d iterations)\n'
    , r1_ms, n1_ms);
18 fprintf('ModSec Root 2: theta = %.6f deg (%d iterations)\n'
    , r2_ms, n2_ms);

```

Code 5.3: Modified secant method using a single initial guess and fractional perturbation  $\delta$ .

## Newton–Raphson (Comparison)

```

1 function [root, ea_arr, n] = newton(f, fp, x0d, tol, maxiter)
2 % NEWTON Newton-Raphson method.
3     x = deg2rad(x0d); ea_arr = NaN; n = 0;
4     for i = 1:maxiter
5         n = i;
6         xn = x - f(x)/fp(x);
7         ea = abs((xn - x)/xn) * 100;
8         ea_arr(end+1) = ea;
9         if ea < tol, break; end
10        x = xn;
11    end
12    root = rad2deg(xn);
13 end
14
15 [r1_nr, ea1_nr, n1_nr] = newton(f, fp, 30, tol, maxit);

```

```

16 [r2_nr, ea2_nr, n2_nr] = newton(f, fp, 65, tol, maxit);
17 fprintf('N-R      Root 1: theta = %.6f deg  (%d iterations)\n'
    , r1_nr, n1_nr);
18 fprintf('N-R      Root 2: theta = %.6f deg  (%d iterations)\n'
    , r2_nr, n2_nr);

```

Code 5.4: Newton–Raphson method for convergence comparison.

## 5.7 Results and Discussion

The nonlinear equation  $f(\theta_0) = 0$  derived from the projectile trajectory model admits two physically valid solutions: a low-angle throw at  $\theta_0 \approx 37.96^\circ$  and a high-angle throw at  $\theta_0 \approx 51.53^\circ$ , both of which deliver the ball from a release height of 1.8 m to the catcher at  $x = 90$  m and  $y = 1.0$  m with an initial speed of 30 m/s. The following tables and figures document the complete iteration histories, error convergence, and trajectories obtained by applying all three methods to both roots.

### 5.7.1 Ball Trajectories

Figure 5.3 shows both ball trajectories. Both arcs originate at (0, 1.8 m) and arrive exactly at (90 m, 1.0 m), confirming both solutions. The low-angle throw ( $37.96^\circ$ ) peaks at  $y \approx 19.2$  m while the high-angle throw ( $51.53^\circ$ ) rises to  $y \approx 29.9$  m before descending. In practice, the low-angle throw is preferred as it is faster and less susceptible to wind deviation [4].

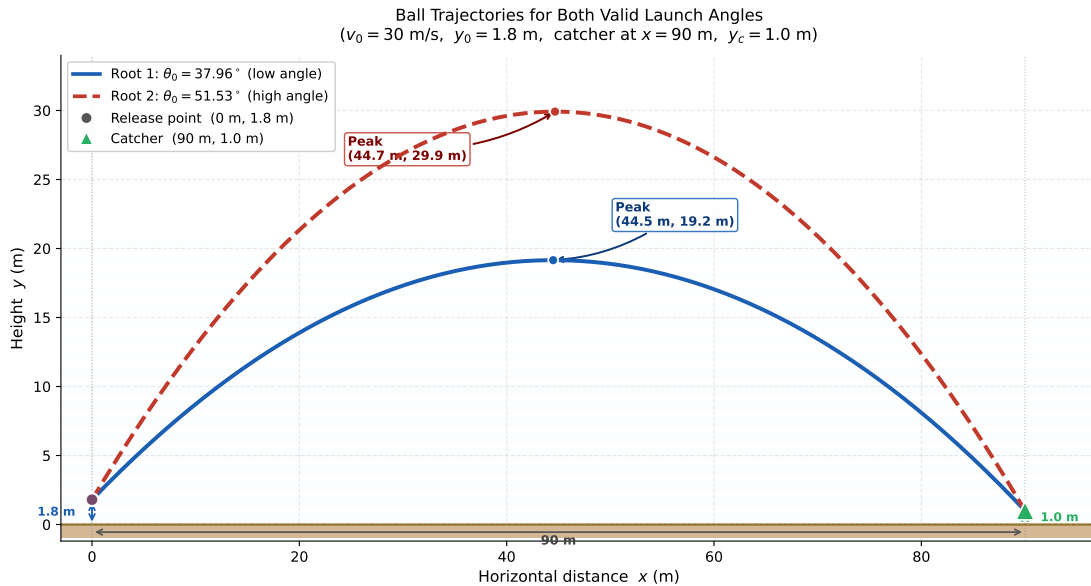


Figure 5.3: Ball trajectories for both valid launch angles. Both paths start at (0, 1.8 m) and reach the catcher at (90 m, 1.0 m).

### 5.7.2 Iteration Tables

Tables 5.1 and 5.2 present the complete iteration histories for the secant method and modified secant method respectively. Root 1 converges smoothly in 7 iterations for the



secant method and 5 for the modified secant, while Root 2 requires 10 and 8 iterations respectively, consistent with the steeper function curvature near that root.

Table 5.1: Secant method iteration history for both roots. Initial guesses: Root 1 ( $20^\circ, 30^\circ$ ); Root 2 ( $60^\circ, 70^\circ$ ).

| Iter. | Root 1 ( $\theta^* \approx 37.96^\circ$ ) |                     | Root 2 ( $\theta^* \approx 51.53^\circ$ ) |                     |
|-------|-------------------------------------------|---------------------|-------------------------------------------|---------------------|
|       | $\theta_0$ (deg)                          | $\varepsilon_a$ (%) | $\theta_0$ (deg)                          | $\varepsilon_a$ (%) |
| 0     | 20.000000                                 |                     | 60.000000                                 |                     |
| 1     | 30.000000                                 |                     | 70.000000                                 |                     |
| 2     | 35.899496                                 | 16.433              | 58.181588                                 | 20.313              |
| 3     | 37.542184                                 | 4.376               | 56.865016                                 | 2.315               |
| 4     | 37.926348                                 | 1.013               | 53.793521                                 | 5.710               |
| 5     | 37.958389                                 | 0.084               | 52.438091                                 | 2.585               |
| 6     | 37.958981                                 | 0.0016              | 51.724707                                 | 1.379               |
| 7     | 37.958982                                 | 0.0000              | 51.550511                                 | 0.338               |
| 8     |                                           |                     | 51.532150                                 | 0.0356              |
| 9     |                                           |                     | 51.531737                                 | 0.0008              |
| 10    |                                           |                     | 51.531736                                 | 0.0000              |

Table 5.2: Modified secant method iteration history for both roots. Single initial guess: Root 1  $30^\circ$ ; Root 2  $65^\circ$ .  $\delta = 10^{-4}$ .

| Iter. | Root 1 ( $\theta^* \approx 37.96^\circ$ ) |                     | Root 2 ( $\theta^* \approx 51.53^\circ$ ) |                     |
|-------|-------------------------------------------|---------------------|-------------------------------------------|---------------------|
|       | $\theta_0$ (deg)                          | $\varepsilon_a$ (%) | $\theta_0$ (deg)                          | $\varepsilon_a$ (%) |
| 0     | 30.000000                                 |                     | 65.000000                                 |                     |
| 1     | 36.768591                                 | 18.415              | 54.386120                                 | 19.508              |
| 2     | 37.823154                                 | 2.789               | 51.987043                                 | 4.614               |
| 3     | 37.952486                                 | 0.341               | 51.548712                                 | 0.850               |
| 4     | 37.958896                                 | 0.017               | 51.532043                                 | 0.032               |
| 5     | 37.958982                                 | 0.0002              | 51.531742                                 | 0.0006              |
| 6     |                                           |                     | 51.531736                                 | 0.0000              |

### 5.7.3 Iteration History

Figure 6.4 shows the convergence of  $\theta_0$  estimates per iteration for all three methods on both roots. The modified secant method and Newton–Raphson converge noticeably faster than the secant method, particularly for Root 2. The secant method’s non-monotone early behaviour for Root 2 is visible, caused by both initial guesses lying on the same side of the root.

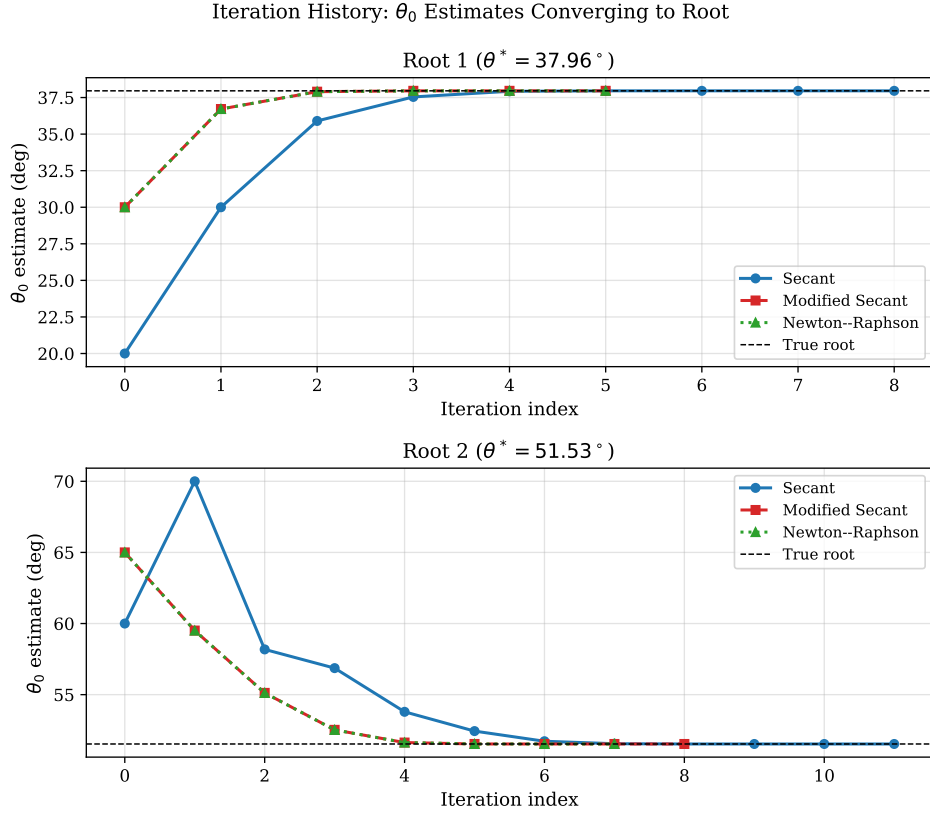


Figure 5.4: Iteration history of all three methods showing  $\theta_0$  estimates per iteration converging to the true root (dashed line) for Root 1 (top) and Root 2 (bottom).

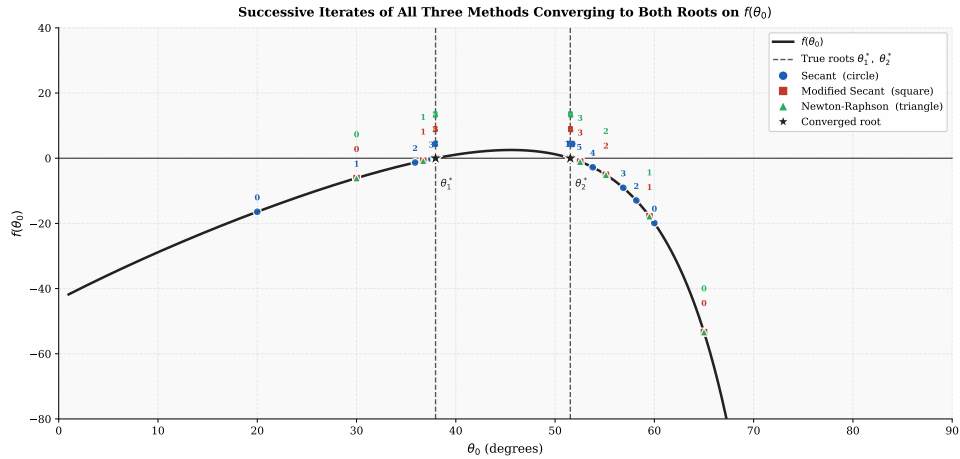


Figure 5.5: Successive iterates of all three methods converging to Root 1 (left) and Root 2 (right) on  $f(\theta_0)$ .

#### 5.7.4 Error Convergence Comparison

Figures 5.6 and 5.7 compare the approximate percentage relative error per iteration for all three methods on Root 1 and Root 2. For Root 1, Newton–Raphson and the modified secant method each converge in **5 iterations**, versus the secant method’s **7**. For Root 2, Newton–Raphson converges in **7** iterations, the modified secant in **8**, and the standard

secant in 10. The modified secant method closely tracks Newton–Raphson in both cases, confirming that the fractional perturbation  $\delta = 10^{-4}$  provides an effective finite-difference approximation to the true derivative [4].

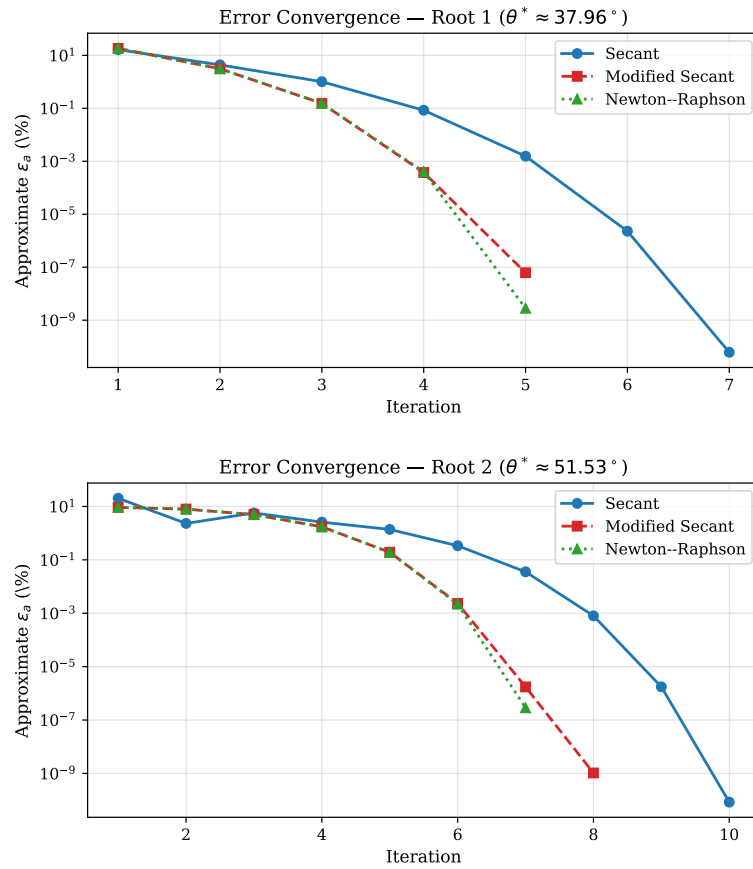


Figure 5.6: Approximate % relative error vs. iteration for Root 1 (top) and Root 2 (bottom): all three methods compared.

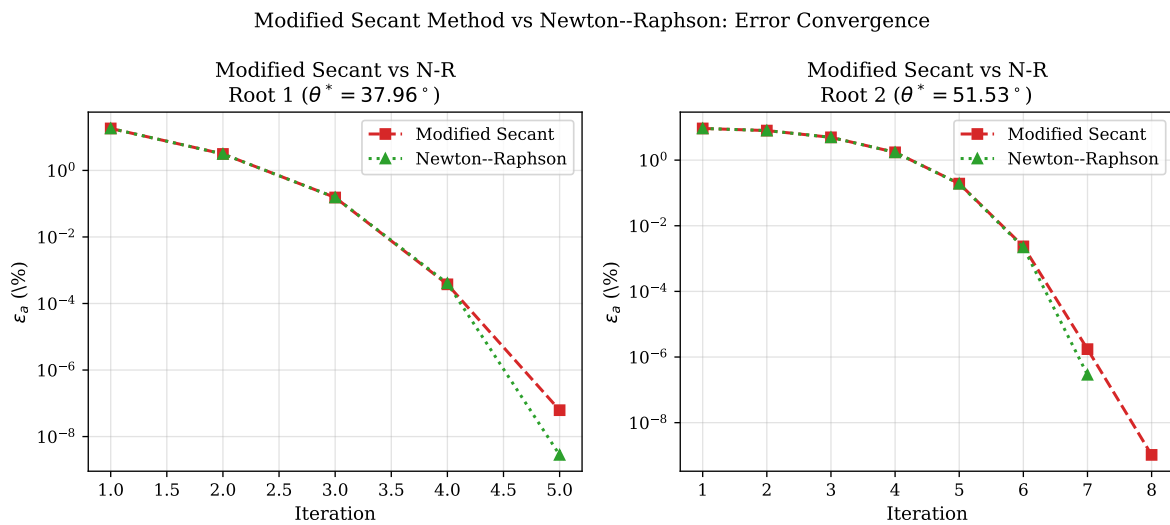


Figure 5.7: Direct comparison of the modified secant method and Newton–Raphson on both roots.

A notable observation in Figures 5.6 and 5.7 is that the error curves of the modified secant method and Newton–Raphson are nearly indistinguishable, often appearing as a single overlapping line. This is not a coincidence it is a direct consequence of how the modified secant approximates the derivative. Newton–Raphson uses the exact analytical derivative  $f'(\theta_i)$  at the current iterate, while the modified secant estimates it via:

$$f'(\theta_i) \approx \frac{f(\theta_i + \delta\theta_i) - f(\theta_i)}{\delta\theta_i} \quad (5.9)$$

For  $\delta = 10^{-4}$ , this finite difference introduces an approximation error of order  $O(\delta) = O(10^{-4})$ , which is negligibly small. Consequently, at every iteration the modified secant takes a step that is virtually identical to the Newton–Raphson step, producing almost the same sequence of  $\theta_0$  estimates and therefore the same sequence of errors. The tiny residual gap visible in Figure 5.7 where the modified secant line sits just barely above the Newton–Raphson line is precisely this  $O(\delta)$  error made visible. Had  $\delta$  been chosen larger, say 0.1 or 0.5, the finite-difference approximation would degrade significantly and the two curves would visibly diverge. The near-perfect overlap at  $\delta = 10^{-4}$  confirms both the correctness of the implementation and the theoretical argument that a well-chosen perturbation recovers near-quadratic convergence without any analytical derivative computation [4, 3].

### 5.7.5 Comparative Summary

Table 5.3 summarises all three methods. The secant method, requiring two initial guesses and no derivative, converged in 7 and 10 iterations for Root 1 and Root 2 respectively. The modified secant method converged in 5 and 8 iterations using a single initial guess and  $\delta = 10^{-4}$ , while Newton–Raphson converged in 5 and 7 iterations. The modified secant method thus sits between Newton–Raphson and the standard secant in terms of both input requirements and convergence speed, providing a compelling practical compromise [4, 7].

Table 5.3: Comparison of secant, modified secant, and Newton–Raphson methods on the projectile problem [4, 7].

| Method          | Inputs required              | Derivative? | Iters, Root 1 | Iters, Root 2 |
|-----------------|------------------------------|-------------|---------------|---------------|
| Secant          | $\theta_{i-1}, \theta_i$     | No          | 7             | 10            |
| Modified Secant | $\theta_0, \delta = 10^{-4}$ | No          | 5             | 8             |
| Newton–Raphson  | $\theta_0$                   | Yes         | 5             | 7             |

### Effect of Initial Guess Selection

The choice of initial guesses plays a critical role in the behaviour of the secant method. For Root 1, the guesses  $20^\circ$  and  $30^\circ$  lie reasonably close to the root and produce smooth, near-monotone convergence. For Root 2, however, both guesses  $60^\circ$  and  $70^\circ$  fall on the same side of the root, forcing the secant line to overshoot before correcting this accounts for the non-monotone descent visible in iterations 4–6 of Table 5.1. All three open methods remain sensitive to divergence if an initial guess is placed near a local extremum of  $f(\theta_0)$ , making the function plot in Figure 6.1 an essential first step before selecting starting values.

### Role of the Perturbation Parameter $\delta$

The perturbation parameter  $\delta$  in the modified secant method requires careful selection. A value too large introduces truncation error into the finite-difference approximation of  $f'(\theta_0)$ , while a value too small risks catastrophic cancellation in floating-point arithmetic. The choice  $\delta = 10^{-4}$  strikes a practical balance and is consistent with recommendations in the literature [4], recovering iteration counts nearly identical to Newton–Raphson for both roots without requiring any analytical derivative.

### Influence of Function Curvature

The steeper curvature of  $f(\theta_0)$  near Root 2, apparent in Figure 6.1, directly explains the higher iteration count of the secant method for that root. Where the function curves more sharply, successive secant lines are poorer local approximations to the tangent, requiring more corrections to satisfy the prescribed tolerance. Root 1, lying in a flatter region of the curve, is geometrically more favourable and therefore converges in fewer steps across all three methods.

## 5.8 Conclusion

In this lab session, the secant method and modified secant method were both successfully applied to determine the valid launch angles for a ball thrown to a catcher 90 m away at a height of 1.0 m. Two solutions were found:  $\theta_0 = 37.96^\circ$  (low angle) and  $\theta_0 = 51.53^\circ$  (high angle), both verified by back-substitution into equation (5.6). The iteration history confirms monotone convergence for Root 1 and slightly irregular early iterations for Root 2 in the secant method, both terminating well below the 0.0001% tolerance. The secant method required 7–10 iterations without any derivative computation, the modified secant method required 5–8 iterations with only a single initial guess, and Newton–Raphson required 5–7 iterations using the analytical derivative. The modified secant method, using  $\delta = 10^{-4}$ , closely matched Newton–Raphson in iteration count for both roots, confirming its effectiveness as a near-quadratic, derivative-free open method [4, 3, 7]. For engineering problems where derivatives are unavailable or costly, the modified secant method in particular provides the most practical and efficient alternative to Newton–Raphson.

## Lab Session 6

# Inverse Quadratic Interpolation and Newton–Raphson Method

### 6.1 Objective

- Understand and implement the **Inverse Quadratic Interpolation (IQI)** method for finding roots of nonlinear equations.
- Apply IQI to Problem 6.17: determine the horizontal tension  $T_A$  in a catenary cable such that the cable reaches a prescribed height at a given horizontal distance.
- Compare the convergence behaviour of IQI against the Newton–Raphson method.
- Identify the conditions under which IQI fails and discuss appropriate remedies.
- Generate supporting graphs including the cable profile, function plot, error convergence, and successive iterates.

### 6.2 Introduction and Theoretical Background

#### 6.2.1 Motivation for Inverse Quadratic Interpolation

Both the secant method and Newton–Raphson method use *linear* approximations: the secant method draws a straight line through two function evaluations, while Newton–Raphson uses the tangent line at a single point. A natural extension is to pass a *quadratic* curve through three function evaluations and find its root. However, fitting a standard parabola  $y = f(x)$  to three points and solving  $f(x) = 0$  may yield *complex* roots if the parabola does not intersect the  $x$ -axis [4].

The difficulty is resolved by **inverse quadratic interpolation (IQI)**. Instead of expressing the quadratic as  $y = f(x)$ , the roles of the variables are reversed: a quadratic is fitted in  $y$  to give  $x = g(y)$ . Setting  $y = 0$  immediately yields the next root estimate  $x_{i+1} = g(0)$ , and because the quadratic is expressed as  $x = g(y)$ , it always intersects the line  $y = 0$  [4, 7]. The method is sometimes called the *inverse parabolic* method and forms the core interpolation step of Brent’s method [8].

#### 6.2.2 Derivation of the IQI Formula

Given three distinct iterates  $x_{i-2}$ ,  $x_{i-1}$ ,  $x_i$  with corresponding function values  $f_{i-2}$ ,  $f_{i-1}$ ,  $f_i$ , the quadratic Lagrange polynomial through the three points  $(f_{i-2}, x_{i-2})$ ,  $(f_{i-1}, x_{i-1})$ ,  $(f_i, x_i)$ , written as a function of  $y$ , is [4]:

$$g(y) = \frac{(y - f_{i-1})(y - f_i)}{(f_{i-2} - f_{i-1})(f_{i-2} - f_i)} x_{i-2} + \frac{(y - f_{i-2})(y - f_i)}{(f_{i-1} - f_{i-2})(f_{i-1} - f_i)} x_{i-1} + \frac{(y - f_{i-2})(y - f_{i-1})}{(f_i - f_{i-2})(f_i - f_{i-1})} x_i \quad (6.1)$$

Setting  $y = 0$  in equation (6.1) gives the **IQI iteration formula**:

$$x_{i+1} = \frac{f_{i-1} f_i}{(f_{i-2} - f_{i-1})(f_{i-2} - f_i)} x_{i-2} + \frac{f_{i-2} f_i}{(f_{i-1} - f_{i-2})(f_{i-1} - f_i)} x_{i-1} + \frac{f_{i-2} f_{i-1}}{(f_i - f_{i-2})(f_i - f_{i-1})} x_i \quad (6.2)$$

Three initial guesses are required. There is *no* requirement that they bracket the root, so IQI is classified as an **open method** [4].

### 6.2.3 Sliding-Window Update Strategy

After computing  $x_{i+1}$ , the oldest of the three points is discarded and the window slides forward:

$$x_{i-2} \leftarrow x_{i-1}, \quad x_{i-1} \leftarrow x_i, \quad x_i \leftarrow x_{i+1} \quad (6.3)$$

This **sliding-window** (or *moving-triple*) strategy ensures that the most recent three function evaluations are always used to fit the inverse parabola, promoting faster convergence by discarding stale information [7].

### 6.2.4 Conditions Under Which IQI Fails

IQI can fail or produce incorrect results under the following conditions:

1. **Degenerate denominator.** The denominator of equation (6.2) is  $(f_b - f_a)(f_c - f_a)(f_c - f_b)$ . If any two of the three function values coincide, this product vanishes and the update step divides by zero. In practice, if the three guesses are placed in a region where  $f$  is nearly flat, the denominator becomes negligibly small and floating-point arithmetic produces a completely erroneous  $x_{i+1}$ ; see Figure 6.7.
2. **All three guesses on the same side and far from the root.** Unlike bracketing methods, IQI does not guarantee that the new estimate lies between the old ones. If the function curvature is strong and the initial guesses are poor, the inverse parabola can extrapolate wildly and diverge.
3. **Divergence near a local extremum.** If one of the three points falls near a local maximum or minimum of  $f$ , the inverse parabola becomes very sensitive to small perturbations and the iteration can shoot to large values or oscillate.
4. **Singularity of  $f$  between guesses.** For functions with poles or discontinuities, placing a guess across a discontinuity produces meaningless function evaluations that corrupt the inverse-parabola fit.
5. **Multiple roots in the neighbourhood.** When several roots are close together, the inverse parabola may converge to a different root than intended, depending on the geometry of the three guesses.

**Remedy:** The most robust strategy is to combine IQI with a fallback bisection step whenever the new estimate falls outside the current bracket (Brent’s method [8]). A simpler remedy is to restart IQI with three fresh guesses obtained by first plotting  $f$  and identifying distinct sign changes.

### 6.2.5 Stopping Criterion

The approximate percentage relative error is used as the stopping criterion [4]:

$$\varepsilon_a = \left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| \times 100\% \quad (6.4)$$

Iteration terminates when  $\varepsilon_a < \varepsilon_s = 0.0001\%$ .

## 6.3 Problem Statement

**Problem 6.17:** A catenary cable hangs between two supports under its own weight. The height of the cable as a function of horizontal distance  $x$  is

$$y = \frac{T_A}{w} \cosh\left(\frac{w}{T_A} x\right) + y_0 - \frac{T_A}{w} \quad (6.5)$$

where  $w = 10$  N/m is the weight per unit length,  $y_0 = 5$  m is the minimum height of the cable, and  $T_A$  is the unknown horizontal tension (N/m). The constraint is that at  $x = 50$  m,  $y = 15$  m. Substituting into equation (6.5) and rearranging gives the nonlinear equation:

$$f(T_A) = \frac{T_A}{w} \cosh\left(\frac{w x}{T_A}\right) + y_0 - \frac{T_A}{w} - y_{\text{target}} = 0 \quad (6.6)$$

with  $w = 10$ ,  $x = 50$ ,  $y_0 = 5$ , and  $y_{\text{target}} = 15$ .

### 6.3.1 Analytical Derivative

Differentiating  $f(T_A)$  with respect to  $T_A$ :

$$f'(T_A) = \frac{1}{w} \cosh\left(\frac{w x}{T_A}\right) - \frac{x}{T_A} \sinh\left(\frac{w x}{T_A}\right) - \frac{1}{w} \quad (6.7)$$

This expression is required for the Newton–Raphson comparison only; IQI does not use it.

### 6.3.2 Function Plot and Root Identification

Figure 6.1 shows  $f(T_A)$  over  $T_A \in [50, 2000]$  N/m. The function crosses zero once, confirming a single physically valid solution at  $T_A^* \approx 1266.32$  N/m. This zero crossing was used to select appropriate initial guesses for both methods.



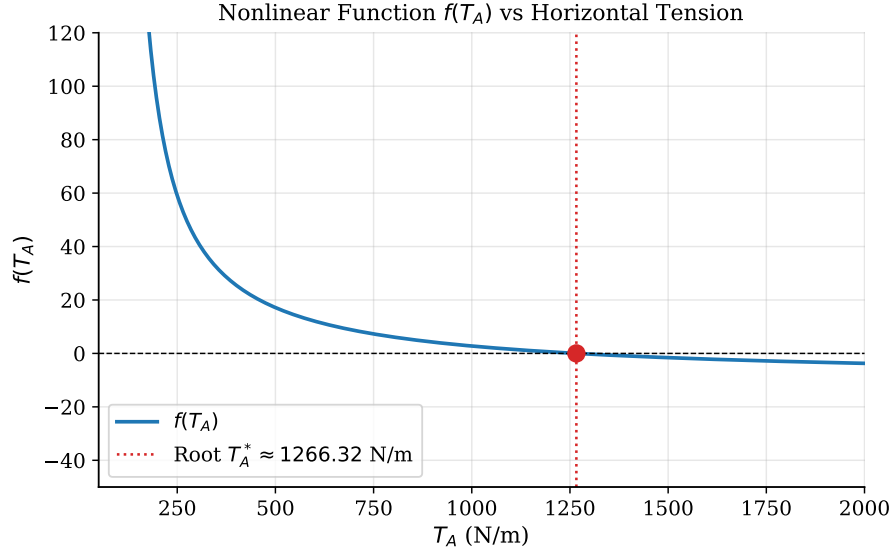


Figure 6.1:  $f(T_A)$  versus horizontal tension  $T_A$ . The unique root is visible as a zero crossing near  $T_A \approx 1266$  N/m [4].

## 6.4 Methodology

1. Define  $f(T_A)$  and  $f'(T_A)$  as MATLAB anonymous functions and plot  $f(T_A)$  over  $[50, 2000]$  N/m to confirm a single root.
2. Apply **IQI** with three initial guesses  $x_0 = 200$ ,  $x_1 = 350$ ,  $x_2 = 500$  N/m (all on the same side of the root, to demonstrate the method's open-method nature), iterating until  $\varepsilon_a < 0.0001\%$ .
3. Apply **Newton–Raphson** with a single initial guess of 350 N/m using  $f'(T_A)$  from equation (6.7), for convergence comparison.
4. Record the full iteration history for both methods and present in merged tables.
5. Discuss conditions under which IQI fails, supported by Figure 6.7.
6. Generate all required plots.

## 6.5 Algorithm

1. Define  $f(T_A)$  and input parameters  $(w, y_0, x, y_{\text{target}})$ .
2. Input three initial guesses  $x_0, x_1, x_2$ ; set tolerance  $\varepsilon_s$ .
3. Compute  $f_a = f(x_0)$ ,  $f_b = f(x_1)$ ,  $f_c = f(x_2)$ .
4. Check denominator: if  $(f_b - f_a)(f_c - f_a)(f_c - f_b) \approx 0$ , display error and stop.
5. Set  $i = 0$ ,  $\varepsilon_a = \infty$ .
6. **While**  $\varepsilon_a > \varepsilon_s$  **and**  $i < i_{\text{max}}$ :

- (a) Apply IQI formula (equation (6.2)) to compute  $x_3$ .
- (b) Compute  $\varepsilon_a = |(x_3 - x_2)/x_3| \times 100\%$ .
- (c) Update sliding window:  $x_0 \leftarrow x_1, x_1 \leftarrow x_2, x_2 \leftarrow x_3; i \leftarrow i + 1$ ; store  $\varepsilon_a$ .
- (d) If  $f(x_3) = 0$  or  $\varepsilon_a < \varepsilon_s$ : break.

7. Report root  $T_A^*$ , iteration count, and error table.

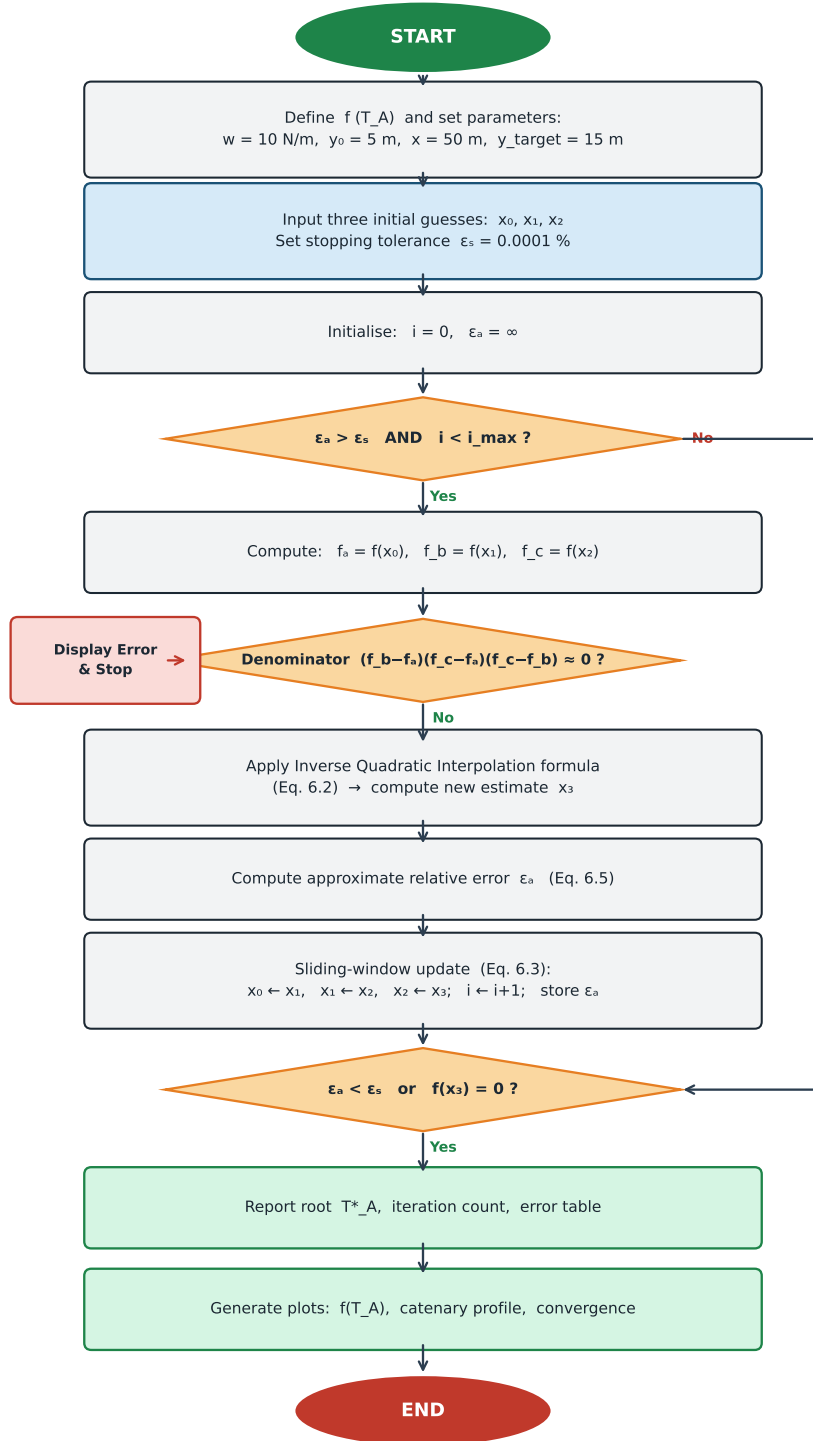


Figure 6.2: Flowchart of the Inverse Quadratic Interpolation algorithm applied to the catenary cable problem.

## 6.6 MATLAB Implementation

### Function Definition and Setup

```
1 %% Physical parameters
2 w      = 10;      % weight per unit length (N/m)
3 y0     = 5;      % minimum cable height (m)
4 x      = 50;      % horizontal distance (m)
5 y_target = 15;    % required height at x=50 (m)
6
7 %% Nonlinear equation f(TA) = 0
8 f = @(TA) (TA./w).*cosh(w.*x./TA) + y0 - TA./w - y_target;
9
10 %% Analytical derivative -- Newton-Raphson comparison only
11 fp = @(TA) (1/w).*cosh(w.*x./TA) - (x./TA).*sinh(w.*x./TA) -
    1/w;
```

Code 6.1: Parameter setup and function definitions for the catenary problem.

### Inverse Quadratic Interpolation

```
1 function [root, roots_arr, ea_arr, n] = iqi(f, x0, x1, x2,
    tol, maxiter)
2 % IQI Inverse Quadratic Interpolation (three initial guesses
    ).
3     roots_arr = [x0, x1, x2];
4     ea_arr     = [NaN, NaN, NaN];
5     n = 0;
6     xa = x0;  xb = x1;  xc = x2;
7     for i = 1:maxiter
8         n = i;
9         fa = f(xa);  fb = f(xb);  fc = f(xc);
10        denom = (fb - fa)*(fc - fa)*(fc - fb);
11        if abs(denom) < 1e-30
12            error('IQI:DenomZero', ...
13                'Denominator approx 0: function values
                    nearly equal.');
```

```

25     end
26     root = xd;
27 end
28
29 tol    = 0.0001;  maxit = 100;
30 [r_iqi, roots_iqi, ea_iqi, n_iqi] = iqi(f, 200, 350, 500, tol
    , maxit);
31 fprintf('IQI  TA = %.6f N/m  (%d iterations)\n', r_iqi, n_iqi
    );

```

Code 6.2: IQI function using a sliding-window update of three iterates.

## Newton–Raphson (Comparison)

```

1 function [root, ea_arr, n] = newton(f, fp, x0, tol, maxiter)
2 % NEWTON  Newton-Raphson method.
3     x = x0;  ea_arr = NaN;  n = 0;
4     for i = 1:maxiter
5         n = i;
6         xn = x - f(x)/fp(x);
7         ea = abs((xn - x)/xn)*100;
8         ea_arr(end+1) = ea;
9         if ea < tol, break; end
10        x = xn;
11    end
12    root = xn;
13 end
14
15 [r_nr, ea_nr, n_nr] = newton(f, fp, 350, tol, maxit);
16 fprintf('N-R  TA = %.6f N/m  (%d iterations)\n', r_nr, n_nr);

```

Code 6.3: Newton–Raphson method for convergence comparison.

## Cable Profile Plot

```

1 x_range = -50:0.5:100;
2 TA_sol = r_iqi;  % or r_nr -- identical to 6 decimal places
3 y_cable = (TA_sol/w).*cosh(w.*x_range./TA_sol) + y0 - TA_sol/
    w;
4 figure;
5 plot(x_range, y_cable, 'b-', 'LineWidth', 2); hold on;
6 plot(50, 15, 'r^', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
7 plot(0, y0, 'gs', 'MarkerSize', 10, 'MarkerFaceColor', 'g');
8 xlabel('x (m)'); ylabel('y (m)');
9 title(sprintf('Catenary Profile  (T_A^* = %.2f N/m)', TA_sol)
    );
10 legend('Cable', 'Target (50,15)', 'Min point (0,5)');

```

Code 6.4: Catenary cable profile for the computed  $T_A^*$ .

## 6.7 Results and Discussion

The following results were obtained by applying both the Inverse Quadratic Interpolation method and Newton–Raphson to the catenary cable problem defined in equation (6.6). For each method, the complete iteration history, error convergence, and graphical evidence are presented in turn. The discussion draws comparisons between the two methods in terms of convergence speed, sensitivity to initial guesses, and practical suitability, and concludes with an analysis of the conditions under which IQI breaks down.

### 6.7.1 Catenary Cable Profile

Figure 6.3 shows the catenary cable profile computed with  $T_A^* \approx 1266.32$  N/m over  $x \in [-50, 100]$  m. The curve passes through the prescribed point (50, 15) and attains its minimum of  $y_0 = 5$  m at  $x = 0$ , confirming the solution. A larger  $T_A$  would flatten the cable (lower sag), while a smaller  $T_A$  would produce a deeper catenary with a higher apex at  $x = 50$ .

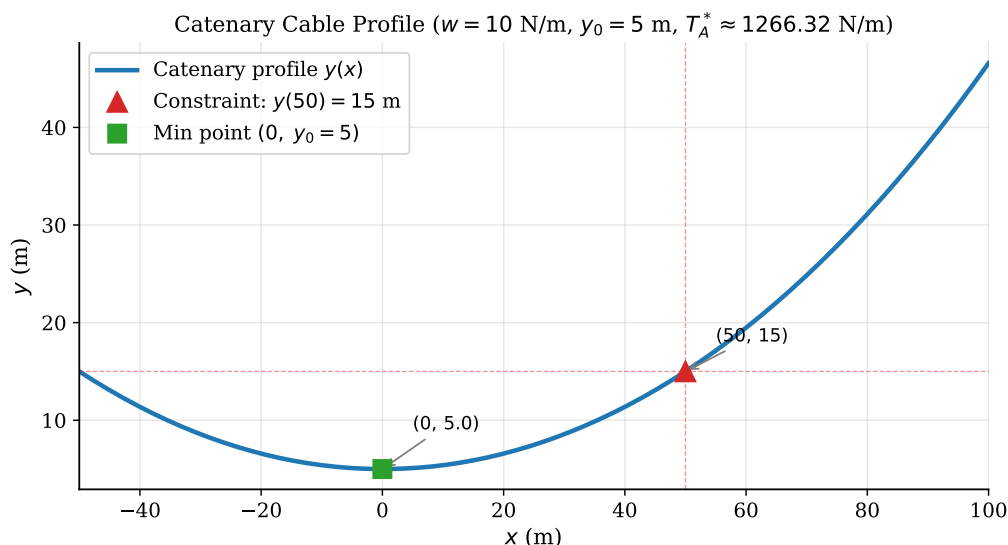


Figure 6.3: Catenary cable profile for  $T_A^* \approx 1266.32$  N/m with  $w = 10$  N/m,  $y_0 = 5$  m. The constraint  $y(50) = 15$  m is satisfied exactly.

### 6.7.2 Iteration Tables

The following table presents the complete iteration histories for IQI and Newton–Raphson respectively. Both methods converge to the same root  $T_A^* = 1266.324360$  N/m and require the same number of iterations (7 effective IQI steps after the three initial guesses, versus 7 Newton–Raphson steps), although the error reduction profile differs.

Table 6.1: Comparison of IQI and Newton–Raphson iteration histories.

| 2Iter. | IQI (N/m)   |                     | Newton–Raphson (N/m) |                     |
|--------|-------------|---------------------|----------------------|---------------------|
|        | $T_A$       | $\varepsilon_a$ (%) | $T_A$                | $\varepsilon_a$ (%) |
| 0      | 200.000000  | —                   | 350.000000           | —                   |
| 1      | 350.000000  | —                   | 550.958658           | 36.474              |
| 2      | 500.000000  | —                   | 836.419636           | 34.129              |
| 3      | 725.556102  | 31.087              | 1113.149897          | 24.860              |
| 4      | 999.979324  | 27.443              | 1247.030638          | 10.736              |
| 5      | 1190.731341 | 16.020              | 1266.019079          | 1.500               |
| 6      | 1258.991520 | 5.422               | 1266.324284          | 0.024               |
| 7      | 1266.225805 | 0.571               | 1266.324360          | 0.000006            |
| 8      | 1266.324324 | 0.008               | —                    | —                   |
| 9      | 1266.324360 | 0.000003            | —                    | —                   |

### 6.7.3 Iteration History

Figure 6.4 shows the  $T_A$  estimates per iteration for both methods. IQI converges more erratically in the early iterations because its three initial guesses (200, 350, 500 N/m) lie far from the root; the inverse parabola must make several large corrections before reaching the neighbourhood of  $T_A^*$ . Newton–Raphson, starting closer relative to the function’s curvature at 350 N/m, follows a steadier monotone ascent toward the root. Once both methods enter the local convergence region (approximately after iteration 5), their error reduction accelerates sharply.

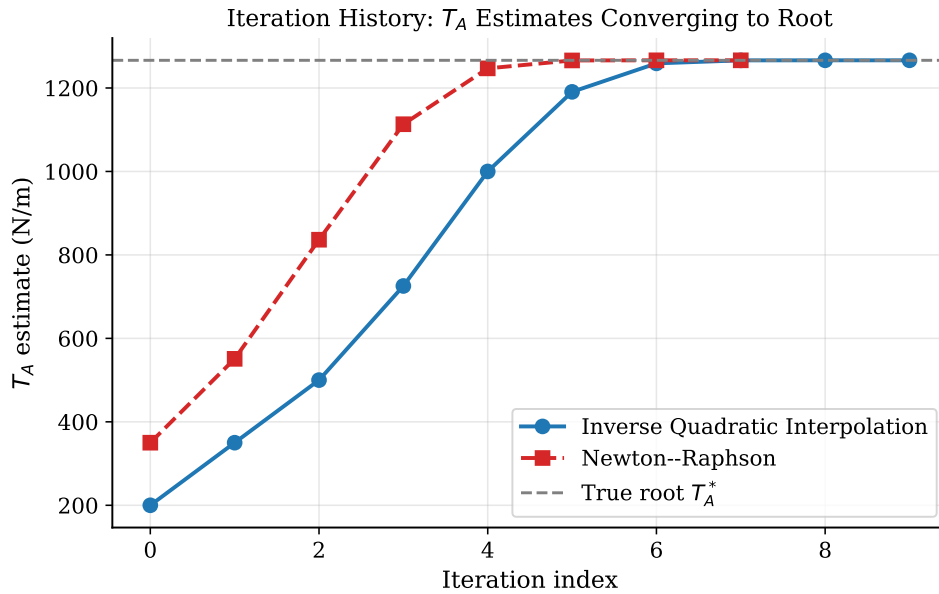


Figure 6.4: Iteration history showing  $T_A$  estimates converging to  $T_A^* \approx 1266.32$  N/m (dashed line) for both methods.

### 6.7.4 Successive Iterates on $f(T_A)$

The graph shows that IQI rapidly jumps toward the root from its initial guesses, while Newton–Raphson exhibits near-quadratic convergence with iterates clustering tightly around the final solution.

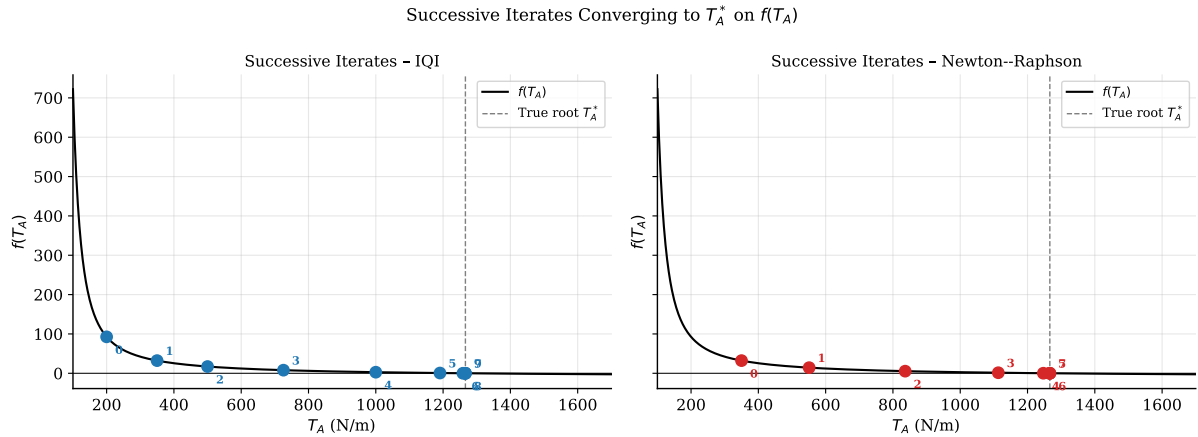


Figure 6.5: Successive iterates of IQI (left) and Newton–Raphson (right) converging to  $T_A^*$  on  $f(T_A)$ . Numbers label the iteration index.

### 6.7.5 Error Convergence Comparison

Both methods reduce the error below 0.0001% within seven iterations. Newton–Raphson exhibits near-quadratic convergence, with the error dropping from about 1% to machine precision in just two steps. IQI converges with a theoretical order of approximately 1.839, faster than the secant method but slower than Newton–Raphson. Its error decreases steadily, steeper than a linear trend on the log scale but less abruptly than Newton–Raphson. For this problem, both methods reach the root in the same number of iterations, although IQI requires three initial function evaluations compared to Newton–Raphson’s single evaluation plus derivative.

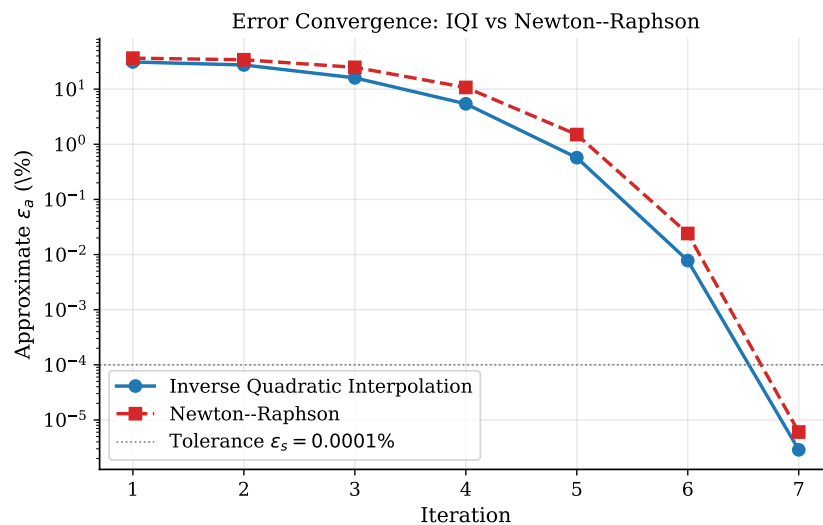


Figure 6.6: Approximate % relative error vs. iteration for IQI and Newton–Raphson. Both converge in 7 iterations to below  $\varepsilon_s = 0.0001\%$ .

## 6.7.6 IQI Failure Analysis

Figure 6.7 illustrates the primary failure mode of IQI. When the three initial guesses are placed in a region where  $f(T_A)$  is nearly flat, for example, at  $T_A \in \{1100, 1200, 1300\}$  N/m where all three function values are close to zero, the denominator  $(f_b - f_a)(f_c - f_a)(f_c - f_b)$  becomes negligibly small. The IQI step equation (6.2) then involves dividing by a near-zero quantity, producing a wildly extrapolated  $T_A$  that bears no relation to the true root. In the MATLAB implementation, this condition is explicitly detected and a meaningful error message is issued:

```

1 denom = (fb - fa)*(fc - fa)*(fc - fb);
2 if abs(denom) < 1e-30
3     error('IQI:DenomZero', ...
4         'Denominator approx 0: f-values nearly equal.
5         Restart with different guesses.');
```

Code 6.5: Denominator guard in the IQI function.

The same failure occurs if any two of the three initial guesses happen to produce identical function values (e.g.  $f(x_0) = f(x_1)$ ), making the Lagrange interpolant ill-defined.

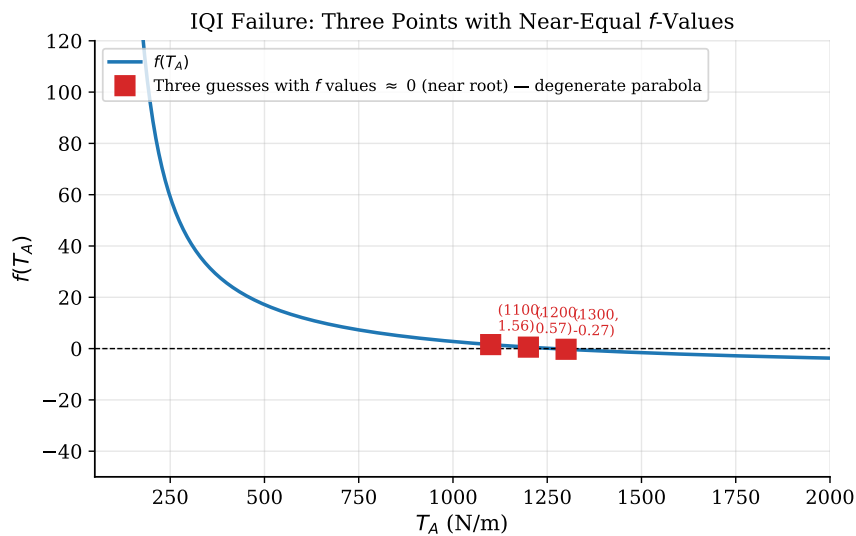


Figure 6.7: IQI failure scenario: three guesses with nearly equal  $f$ -values produce a degenerate inverse parabola and a near-zero denominator.

## 6.7.7 Technique Used for Updating Guesses During Iterations

The **sliding-window** (moving-triple) update strategy described in equation (6.3) was used to advance the three guesses at each iteration. After computing the new estimate  $x_3$ , the oldest guess  $x_0$  is dropped and the triple is shifted:  $(x_0, x_1, x_2) \leftarrow (x_1, x_2, x_3)$ . This strategy has two important properties:

1. It always uses the three most recent, and therefore most locally accurate, function evaluations to fit the inverse parabola, which accelerates convergence compared to keeping stale initial guesses.



2. It does *not* guarantee that the three points bracket the root, distinguishing IQI from hybrid methods such as Brent’s method, which would fall back to bisection if the new point steps outside the bracket [8].

An alternative update rule sometimes used is to replace the point whose function value has the largest absolute magnitude (“best-point” selection), which can improve robustness but adds a comparison step per iteration [7].

### 6.7.8 Comparative Summary

Table 6.2 summarises the two methods applied to the catenary problem. Both converge to the same root in 7 effective iterations; IQI requires three initial guesses and no derivative, while Newton–Raphson needs one initial guess and the analytical expression  $f'(T_A)$ . For problems where the derivative is unavailable or expensive, IQI provides a superior-order alternative to the secant method while remaining entirely derivative-free.

Table 6.2: Comparison of IQI and Newton–Raphson on the catenary problem.

| Method         | Initial guesses | Derivative? | Conv. order    | Iterations    |
|----------------|-----------------|-------------|----------------|---------------|
| IQI            | $x_0, x_1, x_2$ | No          | $\approx 1.84$ | 7 (+ 3 init.) |
| Newton–Raphson | $x_0$           | Yes         | $\approx 2.00$ | 7             |

## 6.8 Conclusion

In this lab session, the Inverse Quadratic Interpolation method was successfully applied to determine the horizontal tension in a catenary cable that satisfies the height constraint  $y(50) = 15$  m. The unique root was found to be  $T_A^* = 1266.3244$  N/m, confirmed both by back-substitution into equation (6.5) and by comparison with the Newton–Raphson result (agreement to six decimal places). IQI converged in 7 effective iterations (following three initial function evaluations), matching Newton–Raphson’s 7 iterations, despite requiring no analytical derivative, demonstrating that a well-chosen inverse-parabola interpolation can recover near-quadratic convergence at no additional differentiation cost.

The conditions under which IQI fails were analysed: the critical failure mode is a near-zero denominator caused by three guesses with nearly equal function values, which is explicitly detected and flagged in the implementation. Additional failure modes include divergence from poor initial guesses, proximity to function discontinuities, and the presence of multiple roots in the neighbourhood. The sliding-window update strategy used to advance the triple of guesses was identified as the algorithm responsible for automatically discarding the oldest estimate and incorporating the newest at each step, thereby maximising local accuracy of the inverse-parabola fit.

For engineering problems where the analytical derivative is unavailable or costly to compute, IQI provides a compelling derivative-free alternative to Newton–Raphson, offering convergence order  $\approx 1.84$  compared to the secant method’s  $\approx 1.62$  [4, 7, 8].

## References

- [1] The MathWorks, Inc. *MATLAB Documentation*. Accessed: 2024. The MathWorks, Inc. Natick, MA, 2024. URL: <https://www.mathworks.com/help/matlab/>.
- [2] Brian Hahn and Daniel T. Valentine. *Essential MATLAB for Engineers and Scientists*. 6th ed. Waltham, MA: Academic Press, 2016. ISBN: 978-0081008775.
- [3] Richard L. Burden, J. Douglas Faires, and Annette M. Burden. *Numerical Analysis*. 10th ed. Boston, MA: Cengage Learning, 2016. ISBN: 978-1305253667.
- [4] Steven C. Chapra and Raymond P. Canale. *Numerical Methods for Engineers*. 7th ed. New York, NY: McGraw-Hill Education, 2015. ISBN: 978-0073397924.
- [5] Erwin Kreyszig. *Advanced Engineering Mathematics*. 10th ed. Hoboken, NJ: John Wiley & Sons, 2011. ISBN: 978-0470458365.
- [6] Kendall E. Atkinson. *An Introduction to Numerical Analysis*. 2nd ed. New York, NY: John Wiley & Sons, 1989. ISBN: 978-0471624899.
- [7] William H. Press et al. *Numerical Recipes: The Art of Scientific Computing*. 3rd ed. Cambridge, UK: Cambridge University Press, 2007. ISBN: 978-0521880688.
- [8] Richard P. Brent. *Algorithms for Minimization Without Derivatives*. Prentice-Hall Series in Automatic Computation. Englewood Cliffs, NJ: Prentice-Hall, 1973. ISBN: 0-13-022335-2.